

Зенит-API

Руководство пользователя.

Оглавление

- [1. Введение](#)
- [2. Установка и настройка](#)
 - [2.1. Системные требования](#)
 - [2.2. Установка дистрибутива для ОС Windows](#)
 - [2.3. Установка для Linux](#)
 - [2.4. Конфигурирование API](#)
- [3. Описание методов Зенит-API](#)
 - [3.1. Авторизация](#)
 - [3.2. Работа с операционными днями](#)
 - [3.2.1. Получение списка открытых операционных дней](#)
 - [3.2.2. Проверка на открытость конкретного операционного дня](#)
 - [3.2.3. Получение состояния операционного дня](#)
 - [3.2.4. Открытие и закрытие операционных дней](#)
 - [3.2.5. Выбор операционного дня](#)
 - [3.3. Входящие документы](#)
 - [3.3.1. Регистрация документа](#)
 - [3.3.1.1. Регистрация документа в формате Формат MFO_09_01](#)
 - [3.3.1.2. Регистрация документа в формате ZOBJ_DOC](#)
 - [3.3.2. Прикрепление XML-файла к документу и его приложениям](#)
 - [3.3.3. Получение информации о документе](#)
 - [3.3.4. Обработка документа](#)
 - [3.3.5. Передача документа в отдел](#)
 - [3.3.6. Ввод услуг в документ](#)
 - [3.3.7. Работа со скан-образами](#)
 - [3.3.7.1. Прикрепление скан-образа к документу](#)
 - [3.3.7.2. Прикрепление скан-образа к приложению](#)
 - [3.3.7.3. Получение информации о скан-образе документа](#)
 - [3.3.7.4. Скачивание скан-образа документа](#)
 - [3.4. Исходящие документы](#)
 - [3.4.1. Формирование автоматических выходных документов \(отчётов\)](#)
 - [3.4.1.1. Исходящий документ без регистрации входящего](#)
 - [3.4.1.1.1. Формирование исходящего документа](#)
 - [3.4.1.1.2. Получение файла исходящего документа](#)
 - [3.4.1.2. Исходящий документ на основе входящего](#)
 - [3.4.1.2.1. Формат файла с входными данными](#)
 - [3.4.1.2.1.1. Пример 1: выписка со счёта \(СВР\)](#)
 - [3.4.1.2.1.2. Пример 2: журнал входящих документов \(СВР\)](#)

- [3.4.1.2.1.3. Пример 3: регистрационный журнал \(ПИФ\)](#)
 - [3.4.1.3. Установка исходящему документу отметки о выдаче](#)
 - [3.4.1.4. Получение исходящих документов, сформированных без использования Зенит-API](#)
 - [3.4.1.5. Создание и скачивание скан-образа в формате PDF](#)
 - [3.4.1.6. Чтение дополнительного файла исходящего документа](#)
 - [3.4.1.7. Получение фильтра типового отчета](#)
 - [3.4.2. Работа с исходящими, полученными из внешнего источника](#)
 - [3.4.2.1. Регистрация исходящих, полученных из внешнего источника](#)
 - [3.4.2.2. Прикрепление файла с данными к исходящему в статусе "На формировании"](#)
 - [3.4.2.3. Перевод документа в статус "Сформирован"](#)
- [3.5. Электронный документооборот](#)
 - [3.5.1. Входящие документы](#)
 - [3.5.1.1. Регистрация входящего документа с указанием СЭД](#)
 - [3.5.1.2. Запуск фоновой обработки входящего документа](#)
 - [3.5.1.3. Чтение списка входящих документов по их идентификаторам](#)
 - [3.5.2. Исходящие документы](#)
 - [3.5.2.1. Чтение списка не выданных исходящих документов](#)
 - [3.5.2.2. Пометка исходящего документа отправленным в СЭД](#)
- [3.6. Проверки ПОД/ФТ](#)
 - [3.6.1. Загрузка списка лиц для проверок ПОД/ФТ](#)
 - [3.6.2. Запуск массовой проверки ПОД/ФТ](#)
- [3.7. Экспорт данных](#)
 - [3.7.1. Журнал экспорта](#)
 - [3.7.2. Экспорт справочников](#)
 - [3.7.3. Экспорт формата состояний](#)
 - [3.7.4. Экспорт исторического формата](#)
 - [3.7.5. Экспорт документа на оплату в бухгалтерию](#)
- [3.8. Импорт данных](#)
 - [3.8.1. Импорт котировок ценных бумаг](#)
- [3.9. Вспомогательные запросы](#)
 - [3.9.1. Получение идентификатора текущего подразделения](#)
 - [3.9.2. Получение количества КД типа "Раскрытие НД" на заданную дату](#)
- [4. Прямые запросы к Зенит-Серверу](#)
 - [4.1. Структура запроса и ответа](#)
 - [4.2. Выбор операционного дня](#)
 - [4.3. Получение данных справочника](#)
 - [4.3.0.1. Чтение по фильтру](#)
 - [4.3.0.2. Чтение по идентификатору](#)

- [4.4. Получение текущей версии и сервис-пака](#)
- [4.5. Чтение данных справочника, изменившихся с контрольной точки](#)
- [4.6. Добавление строки в справочник](#)
- [4.7. Изменение строки справочника](#)
- [4.8. Загрузка файла на сервер](#)
- [4.9. Загрузка файла с сервера](#)
- [4.10. Удаление файла на сервере](#)
- [5. Описание формата MFO_09_01](#)
 - [5.1. Эмитент, ПИФ, управляющая компания](#)
 - [5.1.1. Без указания реестра](#)
 - [5.1.2. Конкретный эмитент](#)
 - [5.1.3. Конкретный ПИФ](#)
 - [5.1.4. По всем ПИФам некоторой УК](#)
 - [5.1.5. Формирование отчета по группе реестров](#)
 - [5.2. Номер и дата документа](#)
 - [5.3. Отправитель входящего документа](#)
 - [5.4. Доставка входящего документа и отправка ответных исходящих](#)
 - [5.4.1. Значение по умолчанию: способ доставки](#)
 - [5.4.2. Значение по умолчанию: место приема и место выдачи](#)
 - [5.5. Тип носителя](#)
 - [5.6. Комментарий и содержимое документа](#)
 - [5.7. Тип формируемого отчета](#)
 - [5.8. Пользовательское наименование отчета](#)
 - [5.9. Присвоение исходящего номера](#)
 - [5.10. Даты отчета](#)
 - [5.10.1. Отчеты без указания дат](#)
 - [5.10.2. Отчеты с одной датой](#)
 - [5.10.3. Отчеты с диапазоном дат](#)
 - [5.11. Зарегистрированное лицо](#)
 - [5.12. Операция](#)
 - [5.13. Фильтр](#)
 - [5.14. Приложения](#)

1. Введение

Сервер Зенит-API представляет собой продукт для организации взаимодействия стороннего программного обеспечения с сервером системы Зенит-Р посредством интерфейса REST API.

С использованием интерфейса Зенит-API можно формировать отчёты, выгружать и загружать реестры, а также выполнять иные операции.

Интерфейс Зенит-API описан в данном руководстве. Примеры запросов запускаются при помощи утилиты cURL.

Обратите внимание, что в данном руководстве `curl`-примеры приведены с переводом строк в длинных запросах для читаемости документации. Использован перевод строк в формате Linux-систем (`\`). Для Windows при необходимости запуска `curl` запускайте утилиту в командной строке (cmd, не powershell) и используйте перевод строки с помощью символа `^` и двойных кавычек.

Например, вызов утилиты curl в командной строке под Windows может выглядеть так:

```
curl -X GET "http://localhost:8080/zenith-object/api/v1/operdays/opened" ^
-H "accept: application/json" ^
-H "Authorization: Basic emVuaXRoLWFwaToxMjM0" ^
-H "X-Zenith-Server: C0"
```

Для более подробного знакомства с эндпоинтами Зенит-API, а также для их тестирования, помимо данной документации рекомендуется использовать Swagger-UI. Он доступен после установки Зенит-API по адресу `http://host:port/zenith-object/docs/`, где `host` и `port` - хост и порт, на которых запущена служба (см. "2. Установка и настройка").

Если Зенит вернул ошибку, которая в API не соответствует какому-либо HTTP статусу, будет возвращен статус `400 Bad Request` с описанием ошибки в теле ответа и кодом ошибки Зенита в заголовке `x-zenith-error-code`.

Также в данном руководстве описан процесс установки и настройки Сервера Зенит-API.

2. Установка и настройка

2.1. Системные требования

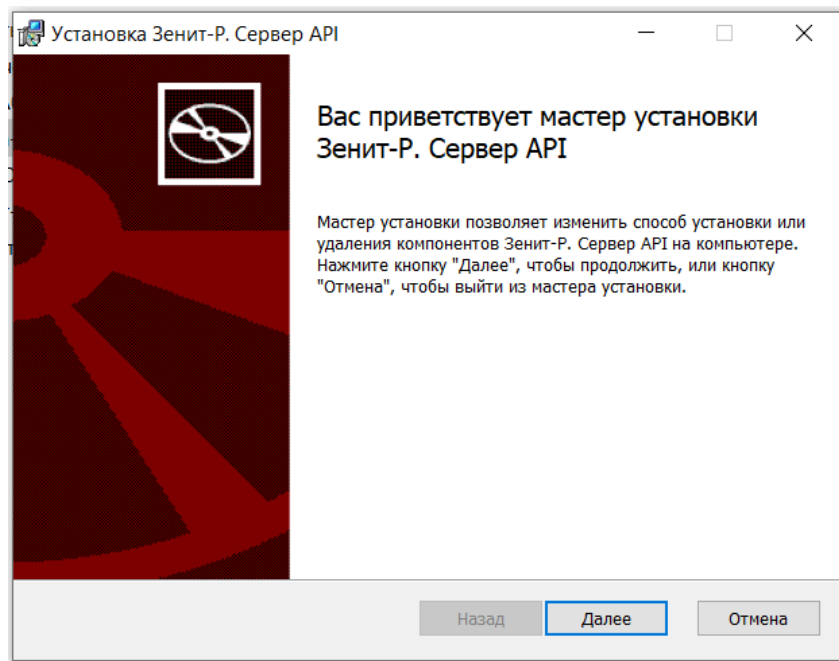
- Операционная система:
 - ОС семейства Windows x64.

- ОС семейства Linux.
- OpenJDK не ниже 11 (для ОС семейства Linux).

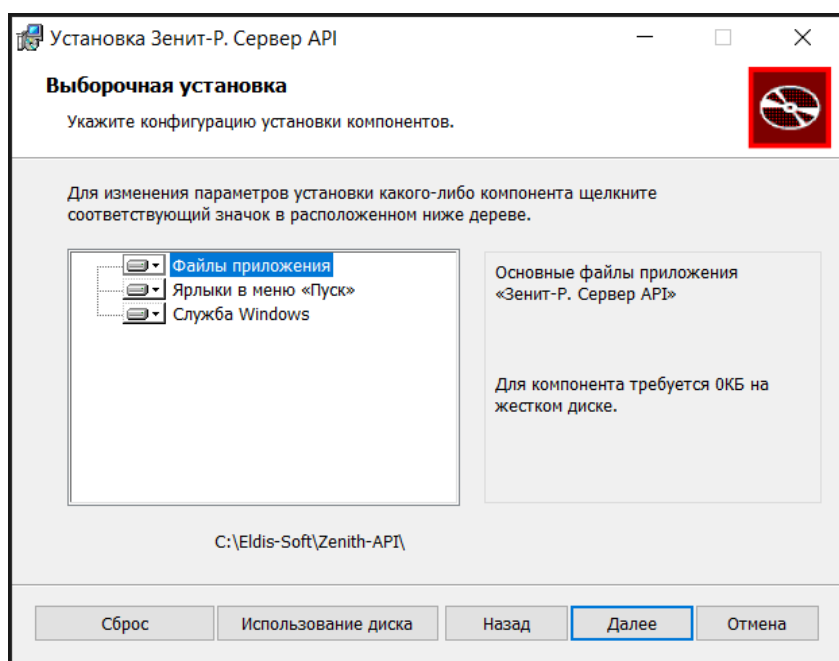
2.2. Установка дистрибутива для ОС Windows

Работа с дистрибутивом в ОС Windows осуществляется следующим образом:

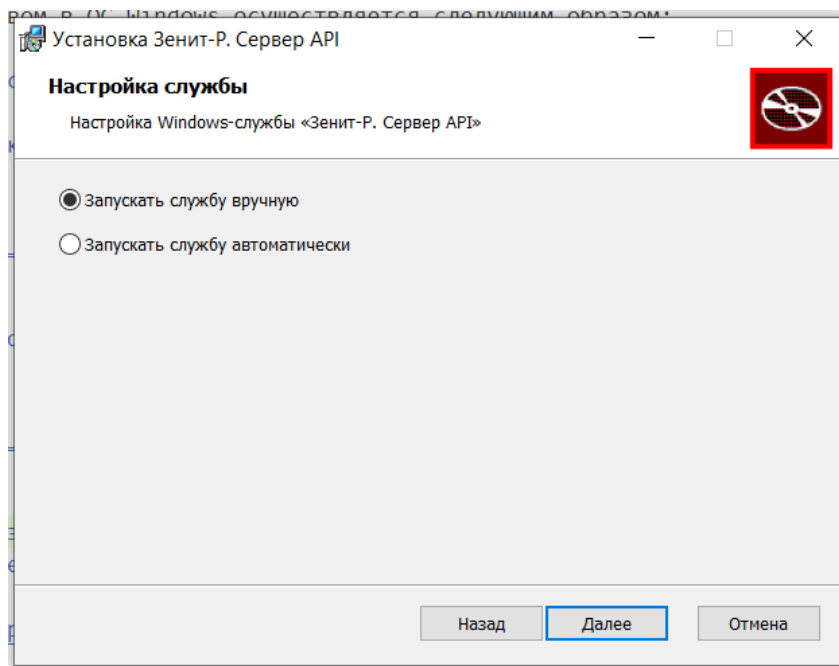
1. Запустите файл установщика Сервера Зенит-API `zenith-api-64.msi`
2. После запуска откроется окно установщика системы. Следуйте инструкциям установщика.



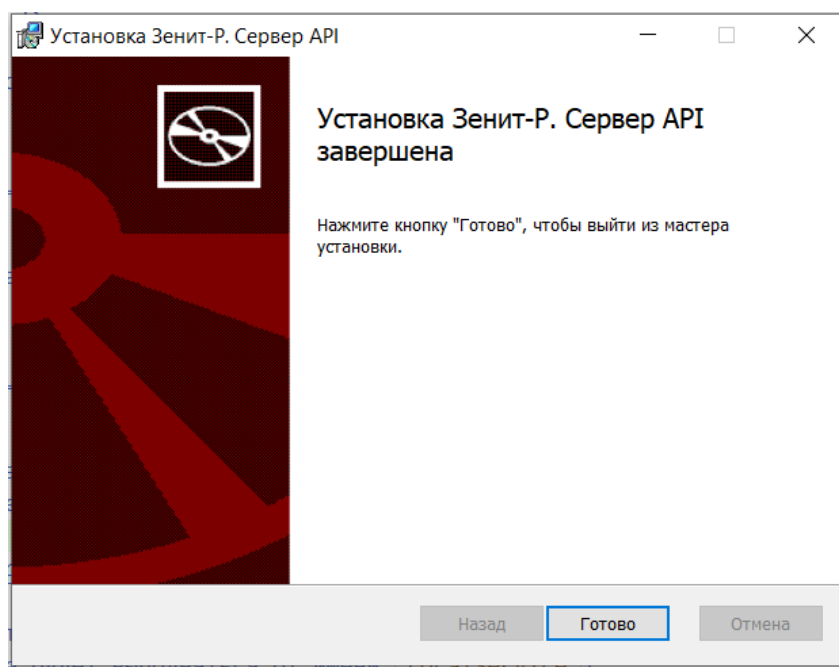
3. Укажите местоположение установки системы



4. Укажите способ запуска службы



5. После успешного завершения установки необходимо создать файл с конфигурацией системы. Подробнее об этом читайте в главе "Конфигурирование API".



6. Попробуйте запустить службу "Сервер Зенит-API".

7. Проверьте логи службы. Они находятся в подкаталоге `logs` каталога установки службы (`launcher.log` - лог запуска службы, `zenith-api-XXXX-XX-XX.log` - лог службы).

2.3. Установка для Linux

Для Linux-систем Сервер Зенит-API распространяется в виде архива с файлами приложения. Процесс настройки выглядит так:

1. Извлеките файлы из архива в удобную вам папку, например, в `/opt/zenith-api`. Далее все пути приводятся относительно этой папки.
2. Создайте файл конфигурации (`conf/app.conf`). Подробнее об этом читайте в главе “Конфигурирование API”.
3. Запустите Сервер Зенит-API (`bin/zenith-api`).
4. Убедитесь, что в логе (`logs/zenith-api-XXXX-XX-XX.log`) при запуске не возникло ошибок.

Настройка операционной системы для автоматического запуска сервера Зенит-API при загрузке зависит от используемой ОС: обратитесь к документации по операционной системе.

2.4. Конфигурирование API

Для конфигурации создайте файл `app.conf` в подкаталоге `conf` каталога с установленной службой. Файл должен быть в кодировке `UTF-8`.

Ниже представлен пример конфигурации с описанием параметров:

```
#####
# Блок параметров HTTP сервера

# Имя/адрес интерфейса, на котором ожидаются подключения по HTTP.
# 0.0.0.0 - использовать все интерфейсы
http.host = "0.0.0.0"
# Порт, на котором ожидаются HTTP подключения
http.port = 8080
# Разрешить работу по HTTPS
http.https.enabled = true
# Имя сервера, которое будет указано в TLS сертификате
http.https.hostname = "zenith-api.reg.ru"
# Порт, на котором ожидаются HTTPS подключения
http.https.port = 8443
# Разрешить работу HTTP при включенном HTTPS
http.https.enablePlainHttp = false
# Пароль для генерации TLS сертификата
http.https.tls.keyPassword = 1234
# Алгоритм TLS сертификата
http.https.tls.alg = RSA_2048_SHA256

#####
# Блок параметров службы УЦ Eldis-CA, требуется если разрешена работа по HTTPS
```



```
# Использовать службу Eldis-CA
ca.enabled = true
# Адрес службы Eldis-CA
ca.url = "http://ca.reg.ru"
# Каталог, в котором будут храниться выданные УЦ ключи и сертификаты TLS
# (Относительные пути вычисляются относительно файла конфигурации)
ca.storagePath = "../tls"

#####
# Список серверов Зенита
servers {
  # Сервер с именем `C0`
  C0 {
    # URL HTTPS сервера
    url = "https://co.reg.ru:8443"
  }
  # Сервер с именем `Subdiv`
  Subdiv {
    # URL HTTPS сервера
    url = "https://subdiv.reg.ru:8443"
  }
}
```

3. Описание методов Зенит-API

3.1. Авторизация

В Зенит-API используется авторизация `Basic Auth`. Параметры авторизации передаются в заголовке запроса `Authorization` в виде пары `имя пользователя:пароль` в закодированном в Base64 виде.

Также необходимо указать имя сервера в заголовке `X-Zenith-Server`. Имя сервера указывается в файле конфигурации сервера Зенит-API.

Вот пример авторизации в заголовке запроса:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/operdays/opened' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

3.2. Работа с операционными днями

3.2.1. Получение списка открытых операционных дней

Вызов эндпоинта `GET /zenith-object/api/v1/operdays/opened` позволяет получить список всех операционных дней, которые открыты на сервере.

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/operdays/opened' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

Возвращается список дат всех открытых на сервере операционных дней. Список отсортирован по возрастанию, т.е. самый старый день первый, самый поздний — последний.

Возвращаются все открытые дни: как те, которые открыты без ограничений, так и дни, открытые по отдельным эмитентам или с ограниченными правами доступа. Права пользователя на доступ к операционному дню не проверяются. Список дней совпадает с тем, который можно увидеть в Зенит-клиенте, в режиме Действия с операционными днями, нажав на кнопку фильтра.

Пример ответа сервера:

```
[
  "2010-12-31",
  "2023-11-09"
]
```

3.2.2. Проверка на открытость конкретного операционного дня

Факт открытости конкретного операционного дня можно проверить вызовом эндпоинта `GET /zenith-object/api/v1/operdays/is_open`, который принимает два параметра:

- `operday` - проверяемый операционный день, например `2024-11-01`. Обязательный параметр;
- `test_rights` - проверка, имеет ли пользователь право на регистрацию и обработку документов этим операционным днем, необязательный параметр, `true/false`.

Строгость проверки:

- `test_rights=true` - проверка, что подключившийся пользователь имеет право регистрировать и обрабатывать документы этим операционным днём;
- `test_rights=false` или не указан - проверка того, что день открыт с любыми правами.

Пример запроса без указания `test_rights`:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/operdays/is_open?operday=2024-11-01' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

Пример запроса с указанием `test_rights`:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/operdays/is_open?operday=2024-11-01&\
  'test_rights=true' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

Закрытый операционный день невозможно выбрать вызовом эндпоинта `POST /zenith-object/api/v1/operdays/set_current` — будет ошибка. Поэтому, если вы не уверены, что день открыт, лучше предварительно проверять это.

3.2.3. Получение состояния операционного дня

Для получения параметров состояния операционного дня используется эндпоинт `GET /zenith-object/api/v1/operdays/{operday}/state`, в обязательном параметре `operday` передается значение проверяемого операционного дня, например `2026-06-17`.

Например:

```
curl -X 'GET' \
  'http://localhost:18080/zenith-object/api/v1/operdays/2026-06-17/state' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

При успешном исполнении запроса сервер возвращает описание состояния операционного дня в виде `json`, формат которого соответствует структуре `OperDayState`:

- `state` - состояние операционного дня, может принимать значения:
 - `Opened` - для открытого операционного дня,
 - `Closed` - для закрытого операционного дня,
- `documentsRegistrationAllowed` - разрешена регистрация входящих и внутренних документов,
- `documentsProcessingAllowed` - разрешена обработка документов (проведений операций в реестре, формирование автоматических исходящих).

- `manualOutgoingDocumentsRegistrationAllowed` - разрешена регистрация ручных исходящих документов.

Параметры `documentsRegistrationAllowed`, `documentsProcessingAllowed`, `manualOutgoingDocumentsRegistrationAllowed` принимают значения `true/false` и возвращаются только для открытых операционных дней.

Например, для закрытого операционного дня тело ответа будет таким:

```
{
  "state": "Closed"
}
```

А для операционного дня, открытого без ограничений, тело ответа будет таким:

```
{
  "state": "Opened",
  "documentsRegistrationAllowed": true,
  "documentsProcessingAllowed": true,
  "manualOutgoingDocumentsRegistrationAllowed": true
}
```

3.2.4. Открытие и закрытие операционных дней

Возможность открывать и закрывать операционные дни введена как инструмент для построения процедуры автоматического перехода на следующий операционный день.

Закрытие операционного дня выполняется вызовом эндпоинта `POST /zenith-object/api/v1/operdays/close` со следующими обязательными параметрами:

- `day_to_close` - какой день нужно закрыть. Возможные значения (согласно часам сервера): - `today` - сегодня, - `yesterday` - вчера, - `all` - все открытые дни;
- `daily_reps` - формировать ли ежедневную отчетность, значения `true/false`;
- `not_daily_reps` - формировать ли отчетность за прочие периоды (еженедельная, ежемесячная), значения `true/false`;
- `event_reps` - формировать ли отчетность по событиям, значения `true/false`.

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/operdays/close?day_to_close='\
  'yesterday&daily_reps=false&not_daily_reps=false&event_reps=false' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

Код возврата `200` означает, что операционный день был закрыт без ошибок.

Для открытия операционного дня необходимо вызвать эндпоинт `POST /zenith-object/api/v1/operdays/open_current`. Этот вызов открывает только новый текущий операционный день, дата которого совпадает с датой на часах сервера. Для открытия старых дней метод не предназначен.

Эндпоинт вызывается с обязательным параметром `only_working_days`, который определяет, требуется открывать только рабочий день или любой, перед его открытием:

- `only_working_days=false` - не проверять, открывать любой день;
- `only_working_days=true` - проверить день по календарю и, если он выходной, не открывать его.

При открытии опередня эндпоинт возвращает код `200`. В случае, если текущий день выходной и `only_working_days=true`, возвращается код `400` с соответствующим описанием ошибки. В заголовке `x-zenith-error-code` приходит код ошибки в Зените.

3.2.5. Выбор операционного дня

Для выбора операционного дня необходимо вызвать эндпоинт `POST /zenith-object/api/v1/operdays/set_current` с параметром `operday` (дата выбираемого дня, например, `2024-11-02`).

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/operdays/set_current?operday=2024-11-02' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

При успешном выборе операционного дня возвращается код `200`. Если выбираемый опередень не открыт, возвращается код `400` с соответствующим сообщением об ошибке. В заголовке `x-zenith-error-code` приходит код ошибки в Зените.

3.3. Входящие документы

Через Зенит-API можно зарегистрировать входящий документ и, либо обработать его, либо передать в какой-нибудь отдел для дальнейшей работы. Если это ЭДО-документ, можно прикрепить его XML-файл.

Данные возможности могут быть полезны как для учёта в Зените документов, реальная работа с которыми идёт где-то в другом месте, так и для автоматической регистрации документов, полученных из внешних источников, дальнейшая работа с которыми будет вестись в Зените.

Данные документа подаются в файле одного из двух форматов:

1. `MF0_09_01` (см. схему `edoscheme\mfo_0901.xsd`). Этот формат прост, но ограничен: в нем можно заполнить только часть полей документа и невозможно описывать поручения.
2. `ZOBJ_DOC` (см. схему `edoscheme\zobj_doc.xsd`). Это формат — клон формата FES-1.0, используемого для обмена документами между Zenith-Порталом и сервером Зенита, с небольшими модификациями. В нём состав данных полнее, поэтому он позволяет подавать заполненные поручения.

Схема `MF0_09_01` описана не только в xsd-файле, но и далее в данном руководстве. Заполняя файл по схеме `ZOBJ_DOC` ориентируйтесь только на xsd-файл, он достаточно подробно документирован.

Возможные действия:

1. Зарегистрировать документ вызовом эндпоинтов `POST /zenith-object/api/v1/documents/register/mfo` (для `MF0_09_01`) или `POST /zenith-object/api/v1/documents/register/zobjdoc` (для `ZOBJ_DOC`).
2. Для ЭДО-документа: прикрепить его XML-файл к документу вызовом эндпоинта `PUT /zenith-object/api/v1/documents/{doc_id}/edoc_file` или к приложению вызовом эндпоинта `PUT /zenith-object/api/v1/documents/{doc_id}/appendices/{appendix_num}/edoc_file`.
3. Прикрепить скан-образ к документу (эндпоинт `PUT /zenith-object/api/v1/documents/{doc_id}/attachment`) или приложению (эндпоинт `PUT /zenith-object/api/v1/documents/{doc_id}/attachment/appendices/{appendix_num}`).
4. Ввести в документ услуги (`PUT /zenith-object/api/v1/documents/{doc_id}/services`), создать и экспортировать документ на оплату (`PUT /zenith-object/api/v1/documents/{doc_id}/bill`).
5. На выбор:
 1. Пометить документ обработанным вызовом эндпоинта `POST /zenith-object/api/v1/documents/by_num_date/process` или `POST /zenith-object/api/v1/documents/by_num_date/process/with_result`.
 2. Или передать документ на обработку в другой отдел вызовом эндпоинта `PUT /zenith-object/api/v1/documents/by_num_date/process/with_result`.
6. Дополнительно можно получить статус и другие данные документа вызовами эндпоинтов `GET /zenith-object/api/v1/documents/{doc_id}/info` или `POST /zenith-object/api/v1/documents/by_num_date/get_info`.

Подробное описание с примерами представлено ниже в этой главе.

3.3.1. Регистрация документа

В системе представлена возможность регистрировать документы в двух форматах - `MFO_09_01` и `ZOBJ_DOC`. Ниже рассмотрены эндпоинты, которые необходимо вызывать при регистрации документов.

Для регистрации документа необходимо предварительно выбрать открытый операционный день.

При успешном исполнении запросов сервер возвращает описание зарегистрированных документов в виде `json`, формат которого соответствует структуре описания документа `DocInfo`:

- `id` - идентификатор документа,
- `uuid` - UUID документа,
- `regnum` - регистрационный номер документа,
- `regdate` - дата регистрации документа,
- `name` - наименование документа,
- `emitent` - код эмитента в виде строки,
- `state` - статус документа, код по внутреннему справочнику "Статус состояния документа",
- `result` - результат обработки, код по внутреннему справочнику "Статус результата".

3.3.1.1. Регистрация документа в формате Формат MFO_09_01

Формальная схема файла, подаваемого серверу Zenit-API для описания документа, находится в файле `mfo_0901.xsd`, который лежит в папке `edoscheme` сервера Зенита. Для ускорения знакомства ниже приведён шаблон, заполняя который вы можете быстро подготовить свои собственные запросы.

```
<?xml version="1.0" encoding="windows-1251"?>
<DOCUMENT>
  <system>REESTR</system>

  <!-- Версия формата. Сейчас допустимо только `MFO_09_01` -->
  <version>MFO_09_01</version>

  <!-- один или два элемента issuer описывают эмитента, ПИФ и УК -->
  <issuer/>

  <!-- общие данные документа -->
  <header/>

  <!-- отправитель документа -->
  <sender/>
```

```

    <!-- как документ принят и как выдавать ответный исходящий -->
    <bearer/>
    <recipient/>
    <agent_point_name/>

    <!-- тип носителя -->
    <media/>

    <!-- комментарий и текстовое описание содержимого документа -->
    <comment/>

    <!-- приложения документа -->
    <appendix/>

    <content/>
</DOCUMENT>

```

Для регистрации документа в формате `MF0_09_01` используйте вызов эндпойнта `POST /zenith-object/api/v1/documents/register/mfo` с обязательным параметром `doc_type`, в котором передается код типа документа по справочнику "Типы документов".

В теле запроса передается xml-описание документа в формате `MF0_09_01`.

Пример запроса:

```

curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/register/mfo?doc_type=6' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="utf-8"?>
<DOCUMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>
  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>13</id>
      </party_id>
    </issuer_name>
  </issuer>
  <header>
    <doc_num>svr503</doc_num>
    <doc_date>
      <datetime>2024-11-01T12:00:30</datetime>
    </doc_date>

```



```

</header>
<appendix>
  <type>32</type>
  <name>Исполнительный лист</name>
  <regdate>2021-11-25</regdate>
  <media>1</media>
  <outnum>1</outnum>
  <docdate>2021-11-25</docdate>
</appendix>
<sender>
  <party_type>1</party_type>
  <party_id>
    <id>5</id>
  </party_id>
  <account_type>4</account_type>
  <individual_or_entity>1</individual_or_entity>
  <name>Открытое акционерное общество "Новосибирскхлебопродукт"</name>
  <short_name>ОАО "НОВОСИБИРСКХЛЕБОПРОДУКТ"</short_name>
  <post_address>
    <plain>
      630089, Новосибирская обл., г. Новосибирск, ул. 1 мая, д.12
    </plain>
  </post_address>
  <entity_reg_dtls>
    <reg_doc_type>
      <entity_reg_doc_type_code>1</entity_reg_doc_type_code>
    </reg_doc_type>
    <reg_num>175-1</reg_num>
    <date_of_incorporation>1993-02-01</date_of_incorporation>
    <reg_org>Новосибирская городская регистрационная палата</reg_org>
  </entity_reg_dtls>
</sender>
<bearer>
  <way>10000</way>
  <abonent>5</abonent>
</bearer>
<recipient>
  <way>004</way>
  <abonent>5</abonent>
</recipient>
<media>
  <id>3</id>
</media>
</DOCUMENT>

```

В ответ сервер присылает описание зарегистрированного документа в формате json, соответствующий структуре описания документа **DocInfo**:

```
{
  "id": "571",
  "uuid": "9c2d1a6d-f21c-47ad-814f-aa0b006cc612",
  "regnum": "ВХ-ЦО-24/0084",
  "regdate": "2024-11-22",
  "name": "Передаточное распоряжение",
  "emitent": "13",
  "state": 1,
  "result": 0
}
```

3.3.1.2. Регистрация документа в формате ZOBJ_DOC

Формальная схема файла, подаваемого серверу Зенит-API для описания документа в формате `ZOBJ_DOC`, находится в файле `zobj_doc.xsd`, который лежит в папке `edoscheme` сервера Зенита.

Для регистрации документа в формате `ZOBJ_DOC` используйте вызов эндпойнта `POST /zenith-object/api/v1/documents/register/zobjdoc`. В теле запроса передается xml-описание документа в формате `ZOBJ_DOC`.

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/register/zobjdoc' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: CO' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="utf-8"?>
<FORM_OF_SHAREHOLDERS version="zenith-obj" uid="fb8f763e-b832-4c00-89a9-0dd123969200">
  <doc_num/>
  <doc_date>
    <date>2024-11-06</date>
  </doc_date>
  <sender>
    <sender_type>01</sender_type>
    <account_number>59</account_number>
    <account_type>OWNER</account_type>
    <person_info>
      <individual>
        <individual_name>Мамедов Гаджимурад Айдинович</individual_name>
        <individual_document>
          <doc_type>
            <individual_doc_type>21</individual_doc_type>
            <narrative>Паспорт</narrative>
          </doc_type>
          <doc_series>53 08</doc_series>
        </individual_document>
      </individual>
    </person_info>
  </sender>
</FORM_OF_SHAREHOLDERS>'
```

```
<doc_num>465987</doc_num>
<doc_date>2008-05-30</doc_date>
<doc_place>УФМС России по Оренбургской области в Северном районе</doc_place>
</individual_document>
</individual>
</person_info>
<post_address>Новосибирск, ул.Вяземского, 56</post_address>
<person_sign>Черепанова А.С.</person_sign>
</sender>
<issuer>
<issuer_id>
<id>5</id>
</issuer_id>
<issuer_info>
<juridical_name>Открытое акционерное общество "Новые технологии"</juridical_name>
<juridical_registration>
<ogrn>
<ogrn>1026601814111</ogrn>
<ogrn_date>2004-03-23</ogrn_date>
<ogrn_place>ФМШ</ogrn_place>
</ogrn>
</juridical_registration>
</issuer_info>
</issuer>
<form_for>ACCH</form_for>
<account_form>
<shareholder_form>
<account_type>OWNER</account_type>
<shareholder_info>
<individual>
<individual_name>Мамедов Гаджимурад Айдинович</individual_name>
<individual_short_name>МАМЕДОВ Г.А.</individual_short_name>
<individual_document>
<doc_type>
<individual_doc_type>21</individual_doc_type>
<narrative>Паспорт</narrative>
</doc_type>
<doc_series>53 08</doc_series>
<doc_num>465987</doc_num>
<doc_date>2008-05-30</doc_date>
<doc_place>УФМС России по Оренбургской области в Северном районе</doc_place>
</individual_document>
<birthtday>1953-04-16</birthtday>
<place_of_birth>г. Ставрополь</place_of_birth>
<nationality>RU</nationality>
<individual_legal_address>
<address_kladr>
<country>RU</country>
<index>355018</index>
<region_number>26</region_number>
<city_type>r</city_type>
```

```

        <city>Ставрополь</city>
        <street_type>пер</street_type>
        <street>Можайский</street>
        <house>12</house>
        <flat>3</flat>
        </address_kladr>
    </individual_legal_address>
    <individual_post_address>
        <address_nostruct>
            <country>RU</country>
            <index>355018</index>
            <address>г. Ставрополь, пер. Можайский, д 12, кв 3</address>
        </address_nostruct>
    </individual_post_address>
</individual>
</shareholder_info>
<distr_div_frm>BANK</distr_div_frm>
<letter_go_type>RLTR</letter_go_type>
<doc_by_post>Yes</doc_by_post>
</shareholder_form>
</account_form>
<appendix>
    <app_type>
        <doc_type_code>PASS</doc_type_code>
        <narrative>Ксерокопия паспорта</narrative>
    </app_type>
    <app_num>1</app_num>
    <app_date>
        <date>2023-12-06</date>
    </app_date>
</appendix>
<add_info>Некий сопроводительный текст, который помещается в поле комментария</add_in
</FORM_OF_SHAREHOLDERS>'

```

В ответ сервер присылает описание зарегистрированного документа в формате json, соответствующий структуре описания документа **DocInfo**:

```

{
  "id": "572",
  "uuid": "b7445a5b-a1bf-4ed1-82cb-0b79aeff9340",
  "regnum": "вх-Ц0-24/0085",
  "regdate": "2024-11-06",
  "name": "Анкета зарегистрированного лица ",
  "emitent": "5",
  "state": 1,
  "result": 0
}

```

3.3.2. Прикрепление XML-файла к документу и его приложениям

Прикрепление XML-файла к документу и его приложениям доступно для всех документов, полученных через ЭДО, зарегистрированных как в формате `ZOBJ_DOC`, так и в формате `MF0_09_01`.

Прикрепление XML-файла к документу осуществляется вызовом эндпоинта `PUT /zenith-object/api/v1/documents/{doc_id}/edoc_file` с обязательными параметрами:

- `doc_id` - идентификатор документа (id в описании документа),
- `msg_type_id` - тип ЭДО-сообщения по справочнику "Типы сообщений ЭДО" подгруппы "Входящие сообщения" и "Сообщения свободного формата", см. в Зенит-клиенте режим "Справочники и системные словари", закладку "Системные и внутренне".

В теле запроса передается прикрепляемый XML-файл.

В примере ниже происходит прикрепление XML-файла к документу, зарегистрированному ранее в примере. Идентификатор документа получен в ответе на запрос регистрации документа `"id": "571"`.

```
curl -X 'PUT' \
  'http://localhost:8080/zenith-object/api/v1/documents/571/edoc_file?msg_type_id=119' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="utf-8"?>
<DOCUMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>
  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>13</id>
      </party_id>
    </issuer_name>
  </issuer>
  <header>
    <doc_num>svr503</doc_num>
    <doc_date>
      <datetime>2024-11-01T12:00:30</datetime>
    </doc_date>
  </header>
```

```

<appendix>
  <type>32</type>
  <name>Исполнительный лист</name>
  <regdate>2021-11-25</regdate>
  <media>1</media>
  <outnum>1</outnum>
  <docdate>2021-11-25</docdate>
</appendix>
<sender>
  <party_type>1</party_type>
  <party_id>
    <id>5</id>
  </party_id>
  <account_type>4</account_type>
  <individual_or_entity>1</individual_or_entity>
  <name>Открытое акционерное общество "Новосибирскхлебопродукт"</name>
  <short_name>ОАО "НОВОСИБИРСКХЛЕБОПРОДУКТ"</short_name>
  <post_address>
    <plain>
      630089, Новосибирская обл., г. Новосибирск, ул. 1 мая, д.12
    </plain>
  </post_address>
  <entity_reg_dtls>
    <reg_doc_type>
      <entity_reg_doc_type_code>1</entity_reg_doc_type_code>
    </reg_doc_type>
    <reg_num>175-1</reg_num>
    <date_of_incorporation>1993-02-01</date_of_incorporation>
    <reg_org>Новосибирская городская регистрационная палата</reg_org>
  </entity_reg_dtls>
</sender>
<bearer>
  <way>10000</way>
  <abonent>5</abonent>
</bearer>
<recipient>
  <way>004</way>
  <abonent>5</abonent>
</recipient>
<media>
  <id>3</id>
</media>
</DOCUMENT>

```

Прикрепление XML-файла к приложению осуществляется вызовом эндпоинта `PUT /zenith-object/api/v1/documents/{doc_id}/appendices/{appendix_num}/edoc_file`, с обязательными параметрами:

- `doc_id` - идентификатор документа (id в описании документа),

- `appendix_num` - номер приложения,
- `msg_type_id` - тип ЭДО-сообщения по справочнику "Типы сообщений ЭДО" подгруппы "Входящие сообщения" и "Сообщения свободного формата", см. в Зенит-клиенте режим "Справочники и системные словари", закладку "Системные и внутренние".

В теле запроса передается прикрепляемый XML-файл.

Пример запроса - прикрепление XML-файла к приложению 1 зарегистрированного выше:

```
curl -X 'PUT' \
  'http://localhost:8080/zenith-object/api/v1/documents/571/appendices/1/edoc_file'\
  '?msg_type_id=32' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="utf-8"?>
<DOCUMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>
  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>13</id>
      </party_id>
    </issuer_name>
  </issuer>
  <header>
    <doc_num>svr503</doc_num>
    <doc_date>
      <datetime>2024-11-01T12:00:30</datetime>
    </doc_date>
  </header>
  <appendix>
    <type>32</type>
    <name>Исполнительный лист</name>
    <regdate>2021-11-25</regdate>
    <media>1</media>
    <outnum>1</outnum>
    <docdate>2021-11-25</docdate>
  </appendix>
  <sender>
    <party_type>1</party_type>
    <party_id>
      <id>5</id>
    </party_id>
```

```

<account_type>4</account_type>
<individual_or_entity>1</individual_or_entity>
<name>Открытое акционерное общество "Новосибирскхлебопродукт"</name>
<short_name>ОАО "НОВОСИБИРСКХЛЕБОПРОДУКТ"</short_name>
<post_address>
  <plain>
    630089, Новосибирская обл., г. Новосибирск, ул. 1 мая, д.12
  </plain>
</post_address>
<entity_reg_dtls>
  <reg_doc_type>
    <entity_reg_doc_type_code>1</entity_reg_doc_type_code>
  </reg_doc_type>
  <reg_num>175-1</reg_num>
  <date_of_incorporation>1993-02-01</date_of_incorporation>
  <reg_org>Новосибирская городская регистрационная палата</reg_org>
</entity_reg_dtls>
</sender>
<bearer>
  <way>10000</way>
  <abonent>5</abonent>
</bearer>
<recipient>
  <way>004</way>
  <abonent>5</abonent>
</recipient>
<media>
  <id>3</id>
</media>
</DOCUMENT>

```

3.3.3. Получение информации о документе

В любой момент после регистрации можно получать некоторую информацию о документе, в том числе его статус, вызвав эндпоинт `GET /zenith-object/api/v1/documents/{doc_id}/info`, где `doc_id` - идентификатор документа, например:

```

curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/documents/571/info' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'

```

В результате успешного выполнения запроса возвращается описание документа в формате `DocInfo` (см. главу "3.3.1. Регистрация документа").

Также можно получить информацию о документе по его номеру и дате регистрации, вызвав эндпоинт `POST /zenith-object/api/v1/documents/by_num_date/get_info` и передав номер и дату регистрации в теле запроса в формате `json`:

```
{
  "num": "Вх-ЦО-24/0084",
  "date": "2024-11-06"
}
```

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/by_num_date/get_info' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "num": "Вх-ЦО-24/0084",
    "date": "2024-11-06"
  }'
```

В результате успешного выполнения запроса также возвращается описание документа в формате `DocInfo`.

Пример такого ответа сервера:

```
{
  "id": "571",
  "uuid": "9c2d1a6d-f21c-47ad-814f-aa0b006cc612",
  "regnum": "Вх-ЦО-24/0084",
  "regdate": "2024-11-06",
  "name": "Передаточное распоряжение",
  "emitent": "13",
  "state": 100,
  "result": 1
}
```

3.3.4. Обработка документа

Обработка документа по номеру и дате регистрации осуществляется вызовом эндпоинта `POST /zenith-object/api/v1/documents/by_num_date/process`. Необязательным параметром запроса передается признак `revoke_on_error` (значения `true/false`), который указывает, аннулировать ли документ при ошибке обработки.

В теле запроса передаются номер и дата регистрации документа в формате `json`, например:

```
{
  "num": "вх-ЦО-24/0084",
  "date": "2024-11-06"
}
```

В примере вызывается обработка документа, зарегистрированного выше, с признаком `revoke_on_error=true`. Номер и дата документа взяты из описания документа `"regnum": "вх-ЦО-24/0084", "regdate": "2024-11-06"`.

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/by_num_date/process'\
  '?revoke_on_error=true' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "num": "вх-ЦО-24/0084",
    "date": "2024-11-06"
  }'
```

Эндпоинт `POST /zenith-object/api/v1/documents/by_num_date/process/with_result` позволяет обработать документ по его номеру и дате регистрации и вернуть данные о сформированном отчете. Его параметры аналогичны параметрам, передаваемым при вызове эндпоинта обработки документа, а в ответ приходит информация о сформированном отчете (см. описание структуры `OutDocLink` в главе 3.4.1.2. Формирование исходящего документа)

Пример запроса:

```
curl -X 'POST' \
  '<http://localhost:8080/zenith-object/api/v1/documents/by_num_date/process/'\
  'with_result?revoke_on_error=false>' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "num": "вх-ЦО-24/0087",
    "date": "2024-11-07"
  }'
```

Пример ответа сервера:

```
{
  "id": "351",
  "regNum": "Ц024-0116",
  "regDate": "2024-11-06",
  "uid": "79cfdceb-57fc-4f3c-b4a3-62329c6ceb77",
  "name": "Выписка со счета зарегистрированного лица"
}
```

3.3.5. Передача документа в отдел

Если вместо обработки необходимо передать документ операторам для дальнейшей работы, необходимо вызвать эндпоинт `PUT /zenith-object/api/v1/documents/{doc_id}/department` с обязательным параметром `doc_id` - идентификатором документа. В теле запроса передается идентификатор отдела. Например, передача документа с идентификатором `570` в отдел `5` выполняется так:

```
curl -X 'PUT' \
  'http://localhost:8080/zenith-object/api/v1/documents/570/department' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '5'
```

3.3.6. Ввод услуг в документ

В зарегистрированный документ можно ввести услуги. Документ не обязательно должен быть только что зарегистрирован через сервер Зенит-API, это может быть любой документ, отвечающий требованиям:

1. Локальный идентификатор документа должен быть известен.
2. Документ должен находиться в папке "Мои документы" авторизовавшего через Зенит-API пользователя.
3. Документ не должен быть аннулированным.

При необходимости перед тем, как ввести услугу в документ, можно получить информацию о документе (см. 3.3.3. Получение информации о документе).

Для добавления услуги в документ необходимо вызвать эндпоинт `POST /zenith-object/api/v1/documents/{doc_id}/services` с обязательным параметром `doc_id` - идентификатором документа. В теле запроса передается описание услуги в виде `json`, соответствующего следующему формату описания услуги `ServiceData`:

- `issuerId` - код реестра (эмитента, ПИФа), к которому относится услуга,

- `serviceTypeId` - код типа услуги по классификатору основных или дополнительных услуг,
- `serviceCount` - количество оказанных услуг,
- `buyer` - покупатель услуги,
- `isProvided` - оказана ли услуга (`true/false`).

Поле `buyer` описывается следующей структурой:

- `type` - тип покупателя. Принимает значения `"issuer"` (эмитент), `"sender"` (отправитель) или `"account"` (зарегистрированное лицо),
- `id` - номер счета, указывается только для покупателей с типом `"account"`.

Пример описания услуги:

```
{
  {
    "issuerId": 5,
    "serviceTypeId": 2,
    "serviceCount": 1,
    "buyer": {
      "type": "account",
      "id": "59"
    },
    "isProvided": false
  }
}
```

Пример добавления услуги в документ с идентификатором `572`:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/548/services' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "issuerId": 5,
    "serviceTypeId": 1,
    "serviceCount": 1,
    "buyer": {
      "type": "issuer"
    },
    "isProvided": false
  }'
```

Регистрироваться могут как основные, так и дополнительные услуги.

Стоимость автоматически рассчитывается по действующему прейскуранту, точно так же, как это происходит при ручном вводе услуги. В теле ответа возвращается полная стоимость услуги в рублях, с включенным НДС (если его ставка ненулевая).

Стоимость возвращается в виде целого числа или десятичной дроби. В дробной части (копейки) не более двух цифр.

3.3.7. Работа со скан-образами

Через Zenith-API можно прикреплять скан-образы как к документу, так и к его вложениям. Допустимо прикрепление как обычного изображения в формате `jpeg`, `tiff`, `bmp` и других, так и файла любого другого типа. Например, `pdf` со сканом или некий архив с дополнительными материалами. Однако, скан-образ всегда представляется одним файлом, несколько файлов прикрепить невозможно. Если это необходимо, запакуйте их в архив.

3.3.7.1. Прикрепление скан-образа к документу

Ко входящему документу можно прикрепить скан-образ. Для этого необходимо вызвать эндпоинт `PUT /zenith-object/api/v1/documents/{doc_id}/attachment` с обязательным параметром `doc_id` - идентификатором документа.

Заголовок запроса `Content-Disposition` должен содержать описание вложения, допускается только тип `attachment`. Имя файла, указанное в описании, сохраняется в Зените.

Сам файл передается в теле запроса.

```
curl -X 'PUT' \
  'http://localhost:8080/zenith-object/api/v1/documents/572/attachment' \
  -H 'accept: */*' \
  -H 'Content-Disposition: attachment; filename*=utf-8\'\'\'\'scan.pdf' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: CO' \
  -H 'Content-Type: application/octet-stream' \
  --data-binary '@scan.pdf'
```

3.3.7.2. Прикрепление скан-образа к приложению

Прикрепление XML-файла к приложению осуществляется вызовом эндпоинта `PUT /zenith-object/api/v1/documents/{doc_id}/attachment/appendices/{appendix_num}`, с обязательными параметрами:

- `doc_id` - идентификатор документа (id в описании документа),

- `appendix_num` - номер приложения.

Заголовок запроса `Content-Disposition` должен содержать описание вложения, допускается только тип `attachment`. Имя файла, указанное в описании, сохраняется в Зените.

Сам файл передается в теле запроса.

Пример такого запроса:

```
curl -X 'PUT' \
  'http://localhost:8080/zenith-object/api/v1/documents/572/attachment/appendices/1' \
  -H 'accept: */*' \
  -H 'Content-Disposition: attachment; filename*=utf-8\'\'\'\'scan.pdf' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/octet-stream' \
  --data-binary '@scan.pdf'
```

3.3.7.3. Получение информации о скан-образе документа

Для получения информации о скан-образе документа необходимо вызвать эндпоинт `GET /zenith-object/api/v1/documents/{doc_id}/scan_info` с обязательным параметром `doc_id` - идентификатором документа.

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/documents/572/scan_info' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В ответ сервер пришлет описание скан-образа документа в виде `json`, формат которого соответствует структуре описания скан-образа `ScanInfo`:

- `formatType` - разновидность скана,
- `fileName` - имя файла,
- `inpNum` - входящий номер,
- `maskedInpNum` - входящий номер, в котором символы, недопустимые для имени файла, заменены на `_`,
- `defaultExtension` - расширение файла,
- `mimeType` - MIME тип.

Разновидность скана приходит в виде числа. Если скан сохраняется в формате Зенита, разновидность может быть следующая:

- `1` — изображение во внутреннем формате Зенита,

- 2 — файл иного типа, например, PDF или DOCX.

Если скан сохранен во внешнем формате, то `formatType=0` (файл сохраняется как есть), другие варианты формата недопустимы.

При отсутствии скана сервер возвращает код 404.

Пример ответа сервера:

```
{
  "formatType": 2,
  "fileName": "скан.jpg",
  "inpNum": "Вх-Ц0-24/0085",
  "maskedInpNum": "Вх-Ц0-24_0085",
  "defaultExtension": "jpg",
  "mimeType": "image/jpeg"
}
```

3.3.7.4. Скачивание скан-образа документа

Для скачивания скан-образа документа необходимо вызвать эндпоинт `GET /zenith-object/api/v1/documents/{doc_id}/scan` с обязательными параметрами `doc_id` - идентификатор документа и `format` - формат скана.

Параметр `format` может принимать следующие значения:

- `pdf`,
- `tiff`,
- `img`,
- `default`.

Значение `default` трактуется как `pdf` для скана во внутреннем формате. Сканы внешнего формата будут возвращаться в их исходном формате.

Пример такого запроса:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/documents/544/scan?format=pdf' \
  -H 'accept: application/octet-stream' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В ответ сервер присылает файл скан-образа в бинарном формате.

3.4. Исходящие документы

3.4.1. Формирование автоматических выходных документов (отчётов)

Формирование выходных документов можно выполнить одним из двух способов:

- с регистрацией входящего документа, на основании которого формируется некоторый отчёт (исходящий документ);
- без регистрации документа, аналогично режиму Зенит-клиента "Формирование отчёта без присвоения исходящего номера".

Второй способ прост, предполагает получение отчёта за один вызов функции и предназначен главным образом для формирования исходящих под собственные нужды. Заполняется минимальный набор полей.

Первый же способ это "полноценное" формирование исходящего документа, с регистрацией входящего документа-основания, с заполнением отправителя, подателя, способа выдачи и с простановкой отметки о выдаче сформированного исходящего.

Рассмотрим эти способы по очереди, начиная с более простого, второго.

3.4.1.1. Исходящий документ без регистрации входящего

3.4.1.1.1. Формирование исходящего документа

Исходящий документ формируется вызовом эндпоинта `POST /zenith-object/api/v1/outgoing_documents/create` со следующими параметрами:

- `data` - информация об отчете в виде `json` формата `ReportData` (описание см.ниже),
- `filter` - файл отчета в формате XML (необязательный параметр).

Информация об отчете `ReportData` содержит следующие поля:

- `outDocType` - код типового исходящего по справочнику "Типы исходящих документов"
- `assignOutDocNum` - присваивать ли отчёту исходящий номер (значения `true/false`)
- `emitent` - код эмитента/ПИФа/группы эмитентов (необязательный параметр). `0` означает "<Не указан>", `-1` - "по всем", по группе эмитентнов - 1000000 (один миллион) + код группы по справочнику,
- `account` - номер счёта ЗЛ (необязательный параметр),
- `beginDate` - дата формирования отчета "от" при периоде формирования отчета (необязательный параметр),
- `endDate` - дата формирования отчета "до" при периоде формирования отчета или единственная при отчете на конкретную дату (необязательный параметр),

- `operationNum` - номер операции по рег. журналу, заполняется при запросе уведомлений о проведении операции (необязательный параметр),
- `corpActionNum` - номер КД/ГО (необязательный параметр).

Пример информации об отчете:

```
{
  "outDocType": 26,
  "assignOutDocNum": true,
  "emitent": -1,
  "endDate": "2024-11-06"
}
```

Пример запроса без указания фильтра отчета:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/create' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: multipart/form-data' \
  -F 'data={
    "outDocType": 26,
    "assignOutDocNum": true,
    "emitent": 13,
    "endDate": "2024-11-06"
  }'
```

Пример запроса с указанием фильтра отчета:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/create' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: multipart/form-data' \
  -F 'data={
    "outDocType": 26,
    "assignOutDocNum": true,
    "emitent": -1,
    "endDate": "2024-11-06"
  }' \
  -F 'filter=@filter.xml;type=text/xml'
```

При успешном формировании отчета сервер возвращает информацию о созданном отчете в виде `json` в формате `OutDocLink`:

- `id` - идентификатор исходящего документа,
- `regNum` - исходящий номер,
- `regDate` - дата регистрации,
- `uid` - универсальный идентификатор исходящего,
- `name` - наименование исходящего документа.

Пример ответа сервера:

```
{
  "id": "347",
  "regNum": "Ц024-0112",
  "regDate": "2024-11-06",
  "uid": "233da92d-47b6-4653-b590-ad739a15f21c",
  "name": "Карта лицевых счетов"
}
```

3.4.1.1.2. Получение файла исходящего документа

Для получения файла сформированного исходящего документа необходимо вызвать эндпоинт `GET /zenith-object/api/v1/outgoing_documents/{out_doc_id}` с обязательными параметрами:

- `out_doc_id` - идентификатор исходящего документа,
- `format` - формат отчета.

Формат отчета может быть описан одним из следующих значений:

- `Docx`,
- `Xlsx` - Excel без шаблона, только данные,
- `Dbf`,
- `Xml`,
- `Pdf`,
- `XlsxWithTemplate` - Excel с учётом настроенного xls-шаблона,
- `UserFormat1`, `UserFormat2`, `UserFormat3`, `UserFormat4` - настраиваемые форматы: первый, второй и т.д. (используются для отчётности ЦБ).

Пример запроса:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/348?format=Pdf' \
  -H 'accept: application/octet-stream' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В ответ сервер присылает файл исходящего документа в бинарном формате.

3.4.1.2. Исходящий документ на основе входящего

Здесь работа состоит из следующих стадий:

1. Подготовка xml-файла с входными данными.
2. Регистрация входящего документа с поручениями на формирование отчётов (см. 3.3.1. Регистрация документа).
3. Обработка входящего документа (см. 3.3.4. Обработка документа), в результате чего происходит формирование отчёта. В результате обработки входящий документ автоматически помещается в архив.
4. При необходимости можно получить файл указанного в запросе сформированного отчёта (см. 3.4.1.1. Получение файла исходящего документа). Последовательными вызовами можно получить один и тот же файл в разных форматах, например, в `DOCX` для представления пользователю и в `XML` для программной обработки. Можно повторить такие вызовы для каждого сформированного отчёта и получить нужные файлы для всех сформированных отчётов.
5. Для каждого из сформированных отчётов устанавливается отметка о выдаче отчёта: или сразу или по факту реальной выдачи отчёта пользователю.

3.4.1.2.1. Формат файла с входными данными

Формальное описание (схема) файла, подаваемого через Zenit-API, находится в файле `mfo_0901.xsd`, который лежит в папке `edoscheme` сервера. Для ускорения знакомства ниже приведён шаблон, с помощью которого можно быстро подготовить необходимые запросы.

```
<?xml version="1.0" encoding="windows-1251"?>
<REQUEST_FOR_STATEMENT>
  <system>REESTR</system>

  <!-- Версия формата. Сейчас допустимо только MFO_09_01 -->
  <version>MFO_09_01</version>

  <!-- 1. Сначала идёт блок, описывающий входящий документ -->

  <!-- один или два элемента issuer описывают эмитента, ПИФ и УК,
        либо группу эмитентов или ПИФов, к которым относится документ
        подробно: 5.1. Эмитент, ПИФ, управляющая компания -->
  <issuer/>

  <!-- общие данные документа
        подробно: 5.2. Номер и дата документа -->
  <header/>

  <!-- отправитель документа
        подробно: 5.3. Отправитель входящего документа -->
```

```
<sender/>

<!-- способ доставки документа и выдачи исходящего
      подробно: 5.4. Доставка входящего документа и отправка ответных исходящих -->
<bearer/>
<recipient/>

<!-- дополнительная информация о месте приёма
      подробно: 5.4. Доставка входящего документа и отправка ответных исходящих -->
<agent_point_name/>

<!-- тип носителя
      подробно: 5.5. Тип носителя -->
<media/>

<!-- комментарий и текстовое описание содержимого документа
      подробно: 5.6. Комментарий и содержимое документа -->
<comment/>
<content/>

<!-- 2. Далее идут данные, относящиеся к исходящему -->

<!-- 2.1 Данные первого отчёта для формирования -->

<!-- код типового исходящего по справочнику Зенита
      "Типы выходных документов"
      подробно: 5.7. Тип формируемого отчёта -->
<statement_type/>

<!-- признак присвоения исходящего номера
      подробно: 5.9. Присвоение исходящего номера -->
<assign_outnum/>

<!-- подробно: 5.10. Даты отчёта -->
<statement>
  <!-- элементы, описывающий даты отчёта -->
  <date/>
  <start_date/>
  <end_date/>

  <!-- дополнительные данные для отчёта 10087 -->
  <quantity/>
</statement>

<!-- если отчёт относится к конкретному ЗЛ, следующие два
      элемента описывают его данные
      подробно: 5.11. Зарегистрированное лицо -->
<account_dtls/>
<account_holder/>

<!-- ссылка на операцию либо по номеру в рег. Журнале, либо по номеру
```

```

        в тех. Журнале; XXX – нужный номер (ссылка может отсутствовать) -->
<operation>
    <!-- Это номер операции в рег. Журнале -->
    <oper_number>XXX</oper_number>
    <!-- если нужна ссылка по номеру в тех. Журнале используйте тэг
        tech_number вместо oper_number -->
</operation>

<!-- фильтр отчёта
        подробно: 5.12. Фильтр -->
<add_info>
    <filter>...</filter>
</add_info>

<!-- 2.2 Список запросов на формирования дополнительных отчётов. -->

<additional_requests>
    <!-- Запрос на формирование дополнительных отчётов. -->
    <additional_request>

        <!-- Данные текущего дополнительного отчёта для формирования -->

        <!-- код типового исходящего по справочнику Зенита
                "Типы выходных документов"
                подробно: 5.7. Тип формируемого отчёта -->
        <statement_type/>

        <!-- признак присвоения исходящего номера
                подробно: 5.9. Присвоение исходящего номера -->
        <assign_outnum/>

        <!-- подробно: 5.10. Даты отчёта -->
        <statement>
            <!-- элементы, описывающий даты отчёта -->
            <date/>
            <start_date/>
            <end_date/>
            <!-- дополнительные данные для отчёта 10087 -->
            <quantity/>
        </statement>

        <!-- если отчёт относится к конкретному ЗЛ, следующие два
                элемента описывают его данные
                подробно: 5.11. Зарегистрированное лицо -->
        <account_dtls/>
        <account_holder/>

        <!-- фильтр отчёта
                подробно: 5.12. Фильтр -->
        <add_info>
            <filter>...</filter>

```

```
</add_info>

</additional_request>
</additional_requests>
</REQUEST_FOR_STATEMENT>
```

В шаблоне дана лишь общая структура сообщения. Подробные описания элементов смотрите в отдельных разделах.

3.4.1.2.1.1. Пример 1: выписка со счёта (СВР)

Пример файла для получения следующих отчётов:

1. Выписки по счёту: Эмитент с кодом 25, выписка на 23.03.2009 по счёту №19.
2. Данные из журнала входящих документов за период с 01.01.2009 по 31.03.2009 по эмитенту с кодом 25.
3. Справка о скупке ценных бумаг по счёту № 3 из эмитента с кодом 25 за период с 01.01.2009 по 31.03.2009 с генерацией исходящего номера.

Отправителем является тот же счёт №19, данные которого заполняются из реестра.

```
<?xml version="1.0" encoding="windows-1251"?>
<REQUEST_FOR_STATEMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>

  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>25</id>
      </party_id>
    </issuer_name>
  </issuer>

  <header>
    <doc_num>216/a</doc_num>
    <doc_date>
      <datetime>2009-03-21T12:00:30</datetime>
    </doc_date>
  </header>

  <sender>
    <party_type>1</party_type>
    <party_id>
      <id>19</id>
    </party_id>
  </sender>
```

```

<statement_type>16</statement_type>
<assign_outnum>1</assign_outnum>

<statement>
  <date>2009-03-23</date>
</statement>

<account_dtls>
  <account_id>
    <id>19</id>
  </account_id>
</account_dtls>

<additional_requests>

  <additional_request>
    <statement_type>21</statement_type>
    <statement>
      <start_date>2009-01-01</start_date>
      <end_date>2009-03-31</end_date>
    </statement>
  </additional_request>

  <additional_request>
    <statement_type>64</statement_type>
    <assign_outnum>1</assign_outnum>

    <statement>
      <start_date>2009-01-01</start_date>
      <end_date>2009-03-31</end_date>
    </statement>
    <account_dtls>
      <account_id>
        <id>3</id>
      </account_id>
    </account_dtls>
  </additional_request>

</additional_requests>
</REQUEST_FOR_STATEMENT>

```

3.4.1.2.1.2. Пример 2: журнал входящих документов (СВР)

Пример файла для получения журнала входящих документов по эмитенту с кодом 6 за период с 01.01.1999 по 23.03.2009. Отправителем является лицо категории "другие", некий Сергеев Е.А. Выходному документу не присваивается номер, вероятно, он запрошен для внутренних нужд.

```

<?xml version="1.0" encoding="windows-1251"?>
<REQUEST_FOR_STATEMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>

  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>6</id>
      </party_id>
    </issuer_name>
  </issuer>

  <header>
    <doc_num>UKWN</doc_num>
    <doc_date>
      <date>1900-01-01</date>
    </doc_date>
  </header>

  <sender>
    <party_type>4</party_type>
    <individual_or_entity>2</individual_or_entity>
    <name>Сергеев Евгений Андреевич</name>
    <short_name>СЕРГЕЕВ Е.А.</short_name>
    <individual_document>
      <doc_type>
        <individual_document_type_code>1</individual_document_type_code>
      </doc_type>
      <doc_ser>12 ET</doc_ser>
      <doc_num>221703</doc_num>
      <doc_date>1988-12-09</doc_date>
      <org>Кировским РОВД г.Новосибирска</org>
    </individual_document>
  </sender>

  <statement_type>21</statement_type>

  <statement>
    <start_date>1999-01-01</start_date>
    <end_date>2009-03-23</end_date>
  </statement>
</REQUEST_FOR_STATEMENT>

```

3.4.1.2.1.3. Пример 3: регистрационный журнал (ПИФ)

Пример файла для получения регистрационного журнала по ПИФ с кодом 159 за период с 01.01.2008 по 31.12.2008. Дополнительный фильтр задаёт следующие ограничения:

- не формировать при отсутствии операций;
- включать только операции, проведённые на основе документов центрального офиса.

```
<?xml version="1.0" encoding="windows-1251"?>
<REQUEST_FOR_STATEMENT>
  <system>REESTR</system>
  <version>MF0_09_01</version>
  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>159</id>
      </party_id>
    </issuer_name>
  </issuer>
  <issuer>
    <issuer_type>1</issuer_type>
    <issuer_name>
      <party_id>
        <id>22</id>
      </party_id>
    </issuer_name>
  </issuer>
  <agent_point_name>
    <agent_name>
      <id>17</id>
    </agent_name>
  </agent_point_name>
  <header>
    <doc_num>217</doc_num>
    <doc_date>
      <date>2009-03-20</date>
    </doc_date>
  </header>
  <statement_type>10065</statement_type>
  <assign_outnum>1</assign_outnum>
  <statement>
    <start_date>2008-01-01</start_date>
    <end_date>2008-12-31</end_date>
  </statement>
  <add_info><filter>
    <COMMON>
      <EXEC_INC>1</EXEC_INC>
      <NONEXEC_INC>0</NONEXEC_INC>
    </COMMON>
  </filter>
</add_info>
</REQUEST_FOR_STATEMENT>
```

```

<OPERNUM_TYPE>1</OPERNUM_TYPE>
<ERR_NO_OPER>1</ERR_NO_OPER>
<OPERSTAT>
  <INC>1</INC>
  <VALUE>0</VALUE>
</OPERSTAT>
<DOCSUBDIV>
  <INC>1</INC>
  <TYPE>0</TYPE>
  <LIST>1</LIST>
</DOCSUBDIV>
</COMMON>
<TOTAL><OPERTYPE>1</OPERTYPE></TOTAL>
</filter> </add_info>
</REQUEST_FOR_STATEMENT>

```

3.4.1.3. Установка исходящему документу отметки о выдаче

Для установки отметки о выдаче без изменений данных выдачи необходимо вызвать эндпоинт `POST /zenith-object/api/v1/outgoing_documents/{out_doc_id}/giveout_mark` с указанием обязательного параметра `out_doc_id` - идентификатора исходящего документа.

```

curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/356/giveout_mark' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''

```

Также можно установить данные выдачи с изменением данных выдачи вызовом эндпоинта `POST /zenith-object/api/v1/outgoing_documents/{out_doc_id}/giveout_mark_with_issuance_data`.

Схема данных документа с данными о выдаче содержится в файле `mfo_0901.xsd` в подпапке `edoscheme` корневой папки Зенит-сервера. Корневой элемент файла с данными о выдаче – `SET_GIVEOUT_DOCMARK`. Пример:

```

<?xml version="1.0" encoding="windows-1251"?>
<SET_GIVEOUT_DOCMARK>
  <system>REESTR</system>
  <!-- Версия формата. Сейчас допустимо только MF0_09_01 -->
  <version>MF0_09_01</version>

  <!-- Блок данных доставки, обязательный для данных выдачи-->
  <delivery>
    <!-- Способ доставки – код по справочнику "Способы

```

```
    доставки документов", обязательный узел для данных выдачи -->
<delivery_type>1</delivery_type>

<!-- Код СЭД по справочнику "Системы ЭДО(СЭД)" -->
<edms>6</edms>

<!--Код абонента ЭДО-->
<edms_user>2</edms_user>

<!-- Код подразделения – места выдачи -->
<agent>1</agent>
</delivery>

<!-- Блок данных получателя -->
<iden>
    <!-- Код типа получателя по справочнику "Тип отправителя". В данном
           примере тип получателя – "Зарегистрированное лицо" -->
    <person_kind>1</person_kind>

    <!-- Тип лица: 1 – ЮЛ, 2 – ФЛ, 3 - ОС -->
    <person_type>1</person_type>

    <!-- Номер счёта в реестре в случае, если указан типа
           получателя – ЗЛ -->
    <account>4</account>

    <!-- Вид счёта в случае, если указан типа получателя – ЗЛ -->
    <account_type>14</account_type>

    <!-- Наименование -->
    <name>ПАО "Труд"</name>

    <!-- Идентификационные данные -->
    <iden2>ОГРН 01234567890001</iden2>
</iden>

<!-- Поле "Кому" -->
<receiving_person>Курьер</receiving_person>

<!-- Почтовый адрес получателя -->
<post_address>Москва, улица Иванова 666, д. 1</post_address>

<!-- Блок данных лица, подписавшего документ -->
<signing_person>
    <!-- ФИО -->
    <name>Иванов Иван Иванович</name>

    <!-- Должность -->
    <position>Главный эксперт</position>
```

```
<!-- Телефон -->
<phone>777-77-77</phone>
</signing_person>
</SET_GIVEOUT_DOCMARK>
```

Если какое-то из полей опущено, оно трактуется как поле с пустым значением: текстовые значения очистятся, числовые занулятся, то, что выбирается из списка, считается не указанным.

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/359/' \
  'giveout_mark_with_issuance_data' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0"?>
<SET_GIVEOUT_DOCMARK>
  <system>REESTR</system>
  <version>MF0_09_01</version>

  <delivery>
    <delivery_type>1</delivery_type>
    <edms>6</edms>
    <edms_user>2</edms_user>
    <agent>1</agent>
  </delivery>

  <iden>
    <person_kind>1</person_kind>
    <person_type>2</person_type>
    <account>4</account>
    <account_type>14</account_type>
    <name>Брагин П.П.</name>
    <iden2>50 02 236341</iden2>
  </iden>

  <receiving_person>Курьер</receiving_person>
  <post_address>ул. Тестовая 1, д. 1</post_address>

  <signing_person>
    <name>Брагин П.П.</name>
    <position>доверенное лицо</position>
    <phone>777-77-77</phone>
  </signing_person>
```

```
</SET_GIVEOUT_DOCMARK>'
```

3.4.1.4. Получение исходящих документов, сформированных без использования Зенит-API

Исходящие, предназначенные для выдачи через Зенит-API могут быть сформированы вручную в Зенит-клиенте или автоматически при закрытии операционного дня. В этом случае их всё равно можно выгрузить через Зенит-API двумя способами:

1. Если известны номер и дата формирования, можно найти исходящий вызовом эндпоинта `POST /zenith-object/api/v1/outgoing_documents/by_num_date/get_info` и затем выгрузить его.

В теле запроса передаются номер и дата исходящего документа в виде `json`

```
{
  "num": "Ц024-0127",
  "date": "2024-11-08"
}
```

Пример такого запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/by_num_date/get_info' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "num": "Ц024-0127",
    "date": "2024-11-08"
  }'
```

В ответ сервер присылает данные об исходящем документе в виде `json` формата `ReportInfo`:

- `id` - идентификатор исходящего,
- `outDocNum` - исходящий номер,
- `reportType` - код типового отчёта по справочнику "Выходные документы",
- `recipientType` - тип места выдачи исходящего по системному справочнику "Типы мест приема/выдачи документов",

- `recipientId` - код места выдачи исходящего по справочнику "Территориальная инфраструктура".

Пример ответа сервера:

```
{
  "id": "361",
  "outDocNum": "Ц024-0127",
  "reportType": 47,
  "recipientType": 0,
  "recipientId": 1
}
```

1. Можно получить полный список невыданных исходящих вызовом эндпоинта `GET /zenith-object/api/v1/outgoing_documents/ungiven_list`.

В запросе необходимо указать следующие параметры:

- `my_documents` - искать только исходящие из папки "Мои документы" (значения `true/false`),
- `creation_date` - дата создания документов, например, `2024-02-14` (необязательный параметр).

На выгружаемые таким образом исходящие действуют ограничения:

- исходящий должен быть сформирован, но не выдан и не передан для выдачи в другое подразделение (статус "Сформирован");
- способом выдачи должно быть указано "Через ZenithObj";
- форматом файла исходящего должен быть бинарный файл (bin; т.е. все виды ручных исходящих, ЭДО-исходящие с файлами xml и служебные исходящие с txt-файлами выгружены быть не могут).

Пример такого запроса:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/' \
  'ungiven_list?my_documents=true' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В ответ сервер присылает список исходящих документе в виде `json`, каждый элемент списка соответствует формату `ReportInfo`:

```
[
  {
    "id": "346",
    "outDocNum": "Ц024-0111",
    "reportType": 26,
    "recipientType": 0,
    "recipientId": 1
  },
  {
    "id": "347",
    "outDocNum": "Ц024-0112",
    "reportType": 26,
    "recipientType": 0,
    "recipientId": 1
  }
]
```

3.4.1.5. Создание и скачивание скан-образа в формате PDF

Для исходящих в бинарном формате (то есть не для ручных и не для ЭДО-исходящих) возможно создавать псевдо-скан-образы в формате PDF. Создание такого "скана" — это приблизительный эквивалент следующих действий:

1. Распечатать исходящий на бумаге
2. Подписать, поставить печать
3. Отсканировать в PDF
4. Прикрепить PDF к исходящему в Зените как скан-образ

Созданный pdf-скан остаётся при исходящем, его всегда можно посмотреть из "Журнала исходящих документов".

Чтобы pdf-скан можно было создать, в шаблоне отчёта должны быть настроены закладки, куда выводится подпись и печать, образы (картинки) печати организации и подписей уполномоченных лиц должны лежать в положенном месте и т.д. (подробнее об этом читайте в файле `pdf\readme.txt` в каталоге Зенит-сервера).

Для создания скан-образа в формате `PDF` необходимо вызвать эндпоинт `POST /zenith-object/api/v1/outgoing_documents/{out_doc_id}/create_scan_from_pdf` со следующими обязательными параметрами:

- `out_doc_id` - идентификатор документа,

- `replace_existing` - признак необходимости заменить существующий скан (значения `true/false`).

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/352/' \
  'create_scan_from_pdf?replace_existing=false' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

В ответ сервер присылает идентификатор скана.

Далее созданный скан можно сказать, вызвав эндпоинт `GET /zenith-object/api/v1/outgoing_documents/scan/{scan_id}` и передав в запросе идентификатор скана, например:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/scan/' \
  '9007199254740991' \
  -H 'accept: application/octet-stream' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

3.4.1.6. Чтение дополнительного файла исходящего документа

По идентификатору исходящего документа можно получить дополнительный файл, прикрепленный к отчету. Для этого используется эндпоинт `GET /zenith-object/api/v1/outgoing_documents/{out_doc_id}/manual_report_file` с обязательным параметром `out_doc_id` - идентификатор документа.

Например:

```
curl -X 'GET' \
  'http://localhost:18080/zenith-object/api/v1/outgoing_documents/1773/manual_report_f' \
  -H 'accept: application/octet-stream' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

Если файл отсутствует, возвращается код `404` с описанием ошибки `У отчета отсутствует дополнительный файл`.

При наличии файла он возвращается в теле ответа, а в заголовках ответа можно прочитать его расширение. Пример ответа:


```
content-type: application/pdf
date: Wed, 17 Jun 2026 11:25:28 GMT
server: Jetty(9.4.50.v20221201)
transfer-encoding: chunked
```

3.4.1.7. Получение фильтра типового отчета

Для получения фильтра типового отчета, нужно вызвать эндпоинт `GET /zenith-object/api/v1/outgoing_documents/{out_doc_type}/filter` с указанием обязательного параметра `out_doc_type` - кода типового отчета.

После успешного выполнения возвращается объект типа `IXMLDOMDocument` с фильтром. Если фильтр у типового отчёта не настроен, то возвращается пустой объект типа `IXMLDOMDocument`.

Пример запроса:

```
curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/10249/filter' \
  -H 'accept: application/xml' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

Пример ответа сервера:

```
<FILTER>
  <ACCTYPE>
    <INC>1</INC>
    <LIST>3;4;5;14</LIST>
  </ACCTYPE>
  <FILTER_VERSION>41</FILTER_VERSION>
  <FILTER_SP>9</FILTER_SP>
</FILTER>
```

3.4.2. Работа с исходящими, полученными из внешнего источника

Если некоторые исходящие документы (отчёты), сформированные вне Зенита, необходимо для каких-то целей зарегистрировать в Зените в "Журнале исходящих документов", то необходимо:

1. Подготовить файл с содержимым исходящего. Это может быть файл Word или Excel, а для ЭДО-исходящих — файл в формате XML. Также файл может отсутствовать.

2. Подготовить файл, описывающий тип исходящего, его получателя и другую информацию.
3. Зарегистрировать исходящий. Исходящий создаётся в статусе "На формировании" и этот статус считается промежуточным. Далее возможны следующие варианты работы с этим документом:
 - если имеется файл с данными, то его можно прикрепить к документу, после чего исходящий документ перейдет в статус "Сформирован",
 - если файла нет, то нужно перевести файл в статус "Сформирован".

При необходимости можно проставить отметку о выдаче (см. 3.4.1.5. Установка исходящему документу отметки о выдаче).

3.4.2.1. Регистрация исходящих, полученных из внешнего источника

Регистрация документа со статусом "На формировании" осуществляется вызовом эндпоинта `POST /zenith-object/api/v1/outgoing_documents/register`, в теле запроса передается XML-файл с описанием документа.

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/register' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<OUTGOING_DOCUMENT>
    <version>MFO_09_01</version>
    <issuer>
      <issuer_type>2</issuer_type>
      <issuer_name>
        <party_id>
          <id>1</id>
        </party_id>
      </issuer_name>
    </issuer>
    <recipient>
      <way>1</way>
      <abonent><id>5</id></abonent>
    </recipient>
    <statement_type>32</statement_type>
    <statement_title>Справка о состоянии счета</statement_title>
    <assign_outnum>1</assign_outnum>
  </OUTGOING_DOCUMENT>'
```

В ответ сервер присылает информацию о зарегистрированном исходящем документе (см. описание структуры `OutDocLink` в главе 3.4.1.2. Формирование исходящего документа) в виде `json`.

Пример ответа:

```
{
  "id": "364",
  "regNum": "Ц024-0130",
  "regDate": "2024-11-06",
  "uid": "0650a4d2-6b5c-41cc-b2dd-67fd4e9d3ae1",
  "name": "Справка о состоянии счета"
}
```

3.4.2.2. Прикрепление файла с данными к исходящему в статусе "На формировании"

Для прикрепления файла с данными необходимо вызвать эндпоинт `POST /zenith-object/api/v1/outgoing_documents/{out_doc_id}/data_file`, где `out_doc_id` - идентификатор исходящего документа, а в теле запроса передать следующие обязательные параметры:

- `data` - информация о файле с данными в виде `json` формата `OutDocFileData`:
 - `dataType` - тип файла с содержимым исходящего. Например, `dataType: {type: "docx"}`. Полный перечень типов перечислен в справочнике "Расширения файла", см. в Зенит-клиенте режим "Справочники и системные словари", закладку "Системные и внутренние". Основные значения:
 - текстовый документ Word 95-2003 (.doc),
 - текстовый документ Word 2007 и позже (.docx),
 - электронная таблица Excel 95-2003 (.xls),
 - электронная таблица Excel 2007 и позже (.xlsx),
 - файл формата XML (.xml).
 - `edoMsgType` - тип ЭДО-сообщения по справочнику "Типы сообщений ЭДО" подгруппы "Входящие сообщения" и "Сообщения свободного формата", см. в Зенит-клиенте режим "Справочники и системные словари", закладку "Системные и внутренние".
- `file` - прикрепляемый файл.

Пример запроса:

```
curl -X 'POST' \
  '<http://localhost:8080/zenith-object/api/v1/outgoing_documents/364/data_file>' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
```

```
-H 'X-Zenith-Server: C0' \  
-H 'Content-Type: multipart/form-data' \  
-F 'data={  
  "dataType": {"type" : "docx"},  
  "edoMsgType": 0  
}' \  
-F 'file=@Zenit0bject.docx;type=application/vnd.openxmlformats-officedocument.wordpr
```

3.4.2.3. Перевод документа в статус "Сформирован"

Для перевода документа в состояние "Сформирован" необходимо вызвать эндпоинт `POST /zenith-object/api/v1/outgoing_documents/{out_doc_id}/formed` , где `out_doc_id` - идентификатор исходящего документа.

Пример запроса:

```
curl -X 'POST' \  
  'http://localhost:8080/zenith-object/api/v1/outgoing_documents/366/formed' \  
-H 'accept: */*' \  
-H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \  
-H 'X-Zenith-Server: C0' \  
-d ''
```

3.5. Электронный документооборот

Для работы с входящими документами в рамках ЭДО используются следующие эндпоинты:

- регистрация документа в поддерживаемом формате для указанной СЭД;
- вызов обработки документа по его идентификатору без ожидания окончания обработки;
- чтение информации о входящих документах по списку идентификаторов с возвратом состояния и идентификаторов сформированных исходящих.

Для работы с исходящими документами в рамках ЭДО используются следующие эндпоинты:

- чтение списка сформированных, но не выданных исходящих для отправки через указанную СЭД;
- пометка исходящего как отправленного в СЭД.

Для установки отметки о выдаче и получения файла исходящего документа необходимо использовать соответствующие эндпоинты, описанные в главе [3.4. Исходящие документы](#)

3.5.1. Входящие документы

3.5.1.1. Регистрация входящего документа с указанием СЭД

Эндпоинт `POST /zenith-object/api/v1/edms/documents/register` позволяет зарегистрировать документ, пришедший из указанной СЭД, в любой поддерживаемой схеме. Принимает параметры:

- `schema` - тип схемы ЭДО, обязательный параметр. Поддерживаемые схемы: `zobj_doc`, `mfo_09_01`;
- `doc_type_id` - тип регистрируемого входящего документа. Если не указан, используется перекодировка,
- `edms_id` - идентификатор СЭД, через которую получен документ,
- `abonent_detection_type` - способ определения абонента ЭДО, если документ получен через СЭД. Принимает следующие значения:
 - `external_software` - всегда указывается спец. абонент "Внешний программный сервис". Такой абонент создается автоматически при необходимости;
 - `search_by_sender` - осуществляется поиск абонента ЭДО по данным отправителя. Если абонент не найден, регистрация завершается с ошибкой;
- `receipt_date` - переопределенная дата приема. При наличии этой даты, Зенит рассчитывает плановую дату обработки от указанной даты, а не от даты регистрации.

В теле запроса передается входящий документ в формате `xml`.

При регистрации документа, в данных документа будет указан способ доставки - через указанную СЭД, с заполненным абонентом. Эти же данные будут указаны в способе выдачи исходящих.

В случае успешной регистрации, в ответ возвращается описание зарегистрированных документов в виде `json`, формат которого соответствует структуре описания документа `DocInfo`:

- `id` - идентификатор документа,
- `uuid` - UUID документа,
- `regnum` - регистрационный номер документа,
- `regdate` - дата регистрации документа,
- `name` - наименование документа,
- `emitent` - код эмитента в виде строки,
- `state` - статус документа, код по внутреннему справочнику "Статус состояния документа",
- `result` - результат обработки, код по внутреннему справочнику "Статус результата".

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/edms/documents/register?schema=mfo_09_0
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="utf-8"?>
<REQUEST_FOR_STATEMENT>
  <system>REESTR</system>
  <version>MFO_09_01</version>

  <issuer>
    <issuer_type>2</issuer_type>
    <issuer_name>
      <party_id>
        <id>5</id>
      </party_id>
    </issuer_name>
  </issuer>

  <header>
    <doc_num>1551</doc_num>
    <doc_date>
      <datetime>2026-06-06T12:00:30</datetime>
    </doc_date>
  </header>

  <sender>
    <party_type>4</party_type>
    <individual_or_entity>2</individual_or_entity>
    <name>Мамедов Гаджимурад Айдинович</name>
    <short_name>МАМЕДОВ Г.А.</short_name>
    <individual_document>
      <doc_type>
        <individual_document_type_code>21</individual_document_type_code>
      </doc_type>
      <doc_ser>53 08</doc_ser>
      <doc_num>465987</doc_num>
      <doc_date>2008-05-30</doc_date>
      <org>УФМС России по Оренбургской области в Северном районе</org>
    </individual_document>
  </sender>

  <statement_type>16</statement_type>
  <assign_outnum>1</assign_outnum>

  <statement>
    <date>2026-05-23</date>
  </statement>
```

```
<account_dtls>
  <account_id>
    <id>59</id>
  </account_id>
</account_dtls>
</REQUEST_FOR_STATEMENT>'
```

Пример ответа:

```
{
  "id": "1867",
  "uuid": "645079fa-4afb-4495-926b-78612a0b71c4",
  "regnum": "вх-Ц0-26/1336",
  "regdate": "2026-06-19",
  "name": "Запрос на выписку",
  "emitent": "5",
  "state": 1,
  "result": 0
}
```

3.5.1.2. Запуск фоновой обработки входящего документа

Для запуска фоновой обработки входящего документа используется эндпоинт `POST /zenith-object/api/v1/edms/documents/{doc_id}/process` с указанием параметров:

- `doc_id` - идентификатор входящего документа. Обязательный параметр,
- `archive` - архивировать документ после обработки, принимает значения `true/false`. Значение по умолчанию `false`.

Вызов является асинхронным, завершается сразу после постановки документа в очередь на обработку, без ожидания ее завершения.

Например:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/edms/documents/1867/process?archive=true' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

3.5.1.3. Чтение списка входящих документов по их идентификаторам

Для чтения списка входящих используется эндпоинт `POST /zenith-object/api/v1/edms/documents/infos/get_by_ids`.

В теле запроса передается список идентификаторов входящих документов в виде массива строк.

Например:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/edms/documents/infos/get_by_ids' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '[
    "1867", "1868"
  ]'
```

В ответ в виде JSON приходит массив объектов структуры `IncomingDocumentWithOutgoingInfo`, каждый из которых описывает входящий документ и связанные с ним исходящие документы.

Описание структуры `IncomingDocumentWithOutgoingInfo` следующее:

- `incoming` - данные входящего документа в виде объекта, соответствующего описанию структуры `IncomingDocumentInfo`, обязательный параметр,
- `outgoing` - список исходящих документов в виде массива объектов соответствующих описанию структуры `OutgoingDocumentInfo`.

Описание структуры `IncomingDocumentInfo` следующее (все поля являются обязательными):

- `id` - идентификатор документа,
- `uuid` - UUID документа,
- `regnum` - входящий номер,
- `regdate` - дата регистрации,
- `regtime` - время регистрации,
- `name` - наименование документа,
- `emitent` - идентификатор эмитента/ПИФ,
- `state` - статус обработки документа, код по внутреннему справочнику "Статус состояния документа",
- `result` - результат обработки документа, код по внутреннему справочнику "Статус результата".

Описание структуры `OutgoingDocumentInfo`:

- `id` - идентификатор документа, обязательный параметр,
- `uuid` - UUID документа, обязательный параметр,

- `outnum` - номер исходящего документа, обязательный параметр,
- `regdate` - дата регистрации, обязательный параметр,
- `regtime` - время регистрации, обязательный параметр,
- `name` - наименование документа, обязательный параметр,
- `generated` - файл исходящего документа сформирован, принимает значения `true/false`, обязательный параметр,
- `fileExtension` - расширение файла сформированного отчета, обязательный параметр,
- `replacementFileExtension` - расширение файла, заменяющего сформированный отчет (при его наличии).

Например:

```
[
  {
    "incoming": {
      "id": "1867",
      "uuid": "645079fa-4afb-4495-926b-78612a0b71c4",
      "regnum": "вх-ЦО-26/1336",
      "regdate": "2026-06-19",
      "regtime": "11:31:00",
      "name": "Запрос на выписку",
      "emitent": "5",
      "state": 100,
      "result": 1
    },
    "outgoing": [
      {
        "id": "1810",
        "uuid": "c7f7919b-fb64-4578-b554-7ba43a341a83",
        "outnum": "ЦО26-1356",
        "regdate": "2026-06-19",
        "regtime": "11:32:00",
        "name": "Выписка со счета зарегистрированного лица",
        "generated": true,
        "fileExtension": "bin"
      }
    ]
  },
  {
    "incoming": {
      "id": "1868",
      "uuid": "ed32f6f8-de39-45b9-b540-27ae1861e4f8",
      "regnum": "вх-ЦО-26/1337",
      "regdate": "2026-06-19",
      "regtime": "11:33:00",
      "name": "Запрос на выписку",
      "emitent": "5",
```

```

    "state": 100,
    "result": 1
  },
  "outgoing": [
    {
      "id": "1811",
      "uuid": "5c25fddb-0cb6-45f0-bf01-c6e1b7afe6e5",
      "outnum": "Ц026-1357",
      "regdate": "2026-06-19",
      "regtime": "11:33:00",
      "name": "Выписка со счета зарегистрированного лица",
      "generated": true,
      "fileExtension": "bin"
    }
  ]
}
]

```

3.5.2. Исходящие документы

3.5.2.1. Чтение списка не выданных исходящих документов

Для чтения списка невыданных документов используется эндпоинт `GET /zenith-object/api/v1/edms/outgoing_documents/unissued`.

Он принимает следующие параметры:

- `edms_id` - идентификатор СЭД,
- `reg_datetime_from` - начало диапазона времени регистрации исходящих (обязательный параметр),
- `reg_datetime_to` - окончания диапазона времени регистрации исходящих (обязательный параметр),
- `limit` - максимальное количество записей (обязательный параметр).

Например:

```

curl -X 'GET' \
  'http://localhost:18080/zenith-object/api/v1/edms/outgoing_documents/unissued?reg_da' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'

```

В ответ приходит список исходящих документов в виде массива объектов соответствующих описанию структуры `OutgoingDocumentInfo` (см. [3.5.1.2. Запуск фоновой обработки входящего документа](#)).

Например:

```
[
  {
    "id": "1810",
    "uuid": "c7f7919b-fb64-4578-b554-7ba43a341a83",
    "outnum": "Ц026-1356",
    "regdate": "2026-06-19",
    "regtime": "11:32:00",
    "name": "Выписка со счета зарегистрированного лица",
    "generated": true,
    "fileExtension": "bin"
  },
  {
    "id": "1811",
    "uuid": "5c25fddb-0cb6-45f0-bf01-c6e1b7afe6e5",
    "outnum": "Ц026-1357",
    "regdate": "2026-06-19",
    "regtime": "11:33:00",
    "name": "Выписка со счета зарегистрированного лица",
    "generated": true,
    "fileExtension": "bin"
  }
]
```

3.5.2.2. Пометка исходящего документа отправленным в СЭД

Для отметки факта отправки исходящего документа в СЭД используется эндпоинт `POST /zenith-object/api/v1/edms/outgoing_documents/{out_doc_id}/mark_as_sent`, где в параметре `out_doc_id` передается идентификатор исходящего документа.

Например:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/edms/outgoing_documents/1810/mark_as_se
-H 'accept: */*' \
-H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
-H 'X-Zenith-Server: C0' \
-d ''
```

При успешной установке документу статуса "Передан в СЭД" возвращается код `200`.

3.6. Проверки ПОД/ФТ

3.6.1. Загрузка списка лиц для проверок ПОД/ФТ

Для загрузки списка лиц для проверок ПОД/ФТ используется эндпоинт `POST /zenith-object/api/v1/opercontrol/person_lists` со следующими обязательными параметрами:

- `file_format` - формат файла, обязательный параметр. Принимает одно из следующих значений:
 - `Excel` - для списков Microsoft Excel,
 - `Csv` - для текстового формата с разделителями,
 - `CftXml` - для XML формата решений MBK (РФМ),
 - `WmdXml` - для XML формата ФРОМУ (РФМ),
 - `UnXml` - для XML формата списков ООН,
 - `TerroristsXml` - для XML формата справочника террористов.
- `list_category` - код по справочнику категорий лиц для контроля. Обязателен для всех форматов, кроме справочника террористов.
- `append` - признак, дополнять ли существующий список. Принимает значение `true/false`. Если не указан, по умолчанию принимает значение `false`.

В теле запроса передается загружаемый файл.

Вызов данного эндпоинта синхронный.

Например:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/opercontrol/person_lists?file_format=Un' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/octet-stream' \
  --data-binary '@consolidated.xml'
```

В ответе возвращается код `200`, если список загрузился успешно.

3.6.2. Запуск массовой проверки ПОД/ФТ

Для запуска массовой проверки ПОД/ФТ используется эндпоинт `POST /zenith-object/api/v1/opercontrol/aml_cft/mass_check`. Вызов эндпоинта без параметров запускает проверку по всем подсистемам и всем эмитентам без установки признака периодической проверки. Также при необходимости можно указать следующие параметры проверки:

- `subsystem` - подсистема, для реестров которой выполнить проверку. Не требуется, если указан параметр `emitent_id`, либо если проверка выполняется по всем подсистемам. Принимает значение `Srv/Pif`;
- `emitent_id` - идентификатор эмитента или ПИФ. Не требуется, если указан параметр `subsystem`,
- `periodic` - периодическая проверка. Принимает значения `true/false`, по умолчанию `false`.

Вызов данного эндпоинта синхронный.

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/opercontrol/aml_cft/mass_check?subsystem' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

3.7. Экспорт данных

Подробная документация по экспорту находится в файле `ZenithExim.doc`, являющимся частью документации к Зениту. Там введены такие понятия как формат экспорта и шаблон данных.

Результатом экспорта всегда является набор таблиц в формате `dbf`, содержащий реляционные данные. Это основной массив информации.

В некоторых форматах выгружается дополнительный набор файлов, с такими данными, как сканированные образы документов или содержимое выходных документов.

Dbf-файлы всегда выгружаются в кодировке `866`, так же известной как `CP-866`, `DOS Cyrillic`, `Cyrillic OEM`. Говоря неформально, формат задаёт, что выгружается, а шаблон определяет колоночный состав выгружаемых данных.

С технической точки зрения, шаблон представляет собой набор пустых dbf-файлов, которые передаются на сервер, заполняются там данными и возвращаются в качестве результата. Шаблон можно подготовить при помощи модуля экспорта-импорта ("Зенит-Р Экспорт-Импорт"), который создаёт dbf-файлы с полным составом колонок.

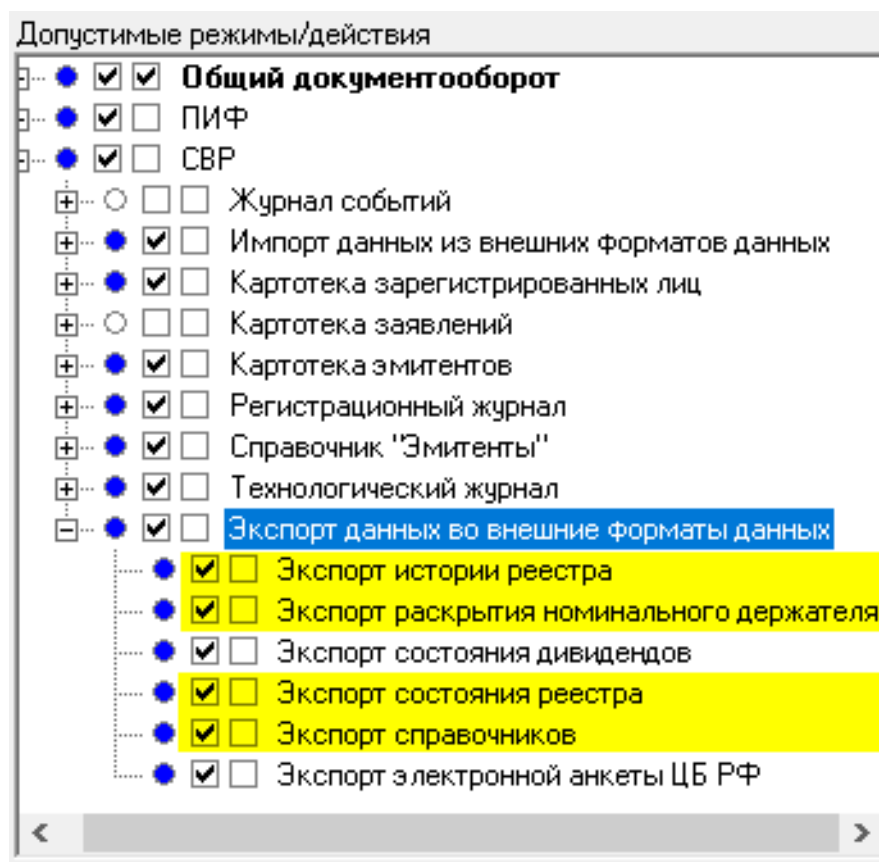
Как правило, удалять колонки нет необходимости, однако это возможно при помощи любой программы, позволяющей редактировать структуру dbf-файла.

Отметим, что добавление неизвестных Зениту колонок не ведёт к негативным последствиям, они просто остаются незаполненными и могут использоваться вами для хранения дополнительных данных.

Через Зенит-API можно экспортировать данные в следующих форматах:

- справочники;
- состояние (реестра);
- исторический (история одного реестра).

Доступ к различным форматам регламентируется правами пользователя. Для успешного экспорта необходимо убедиться, что пользователь, от имени которого работает сервер Зенит-API, имеет следующие права (см. режим "Управление пользователями", закладку "Сотрудники"):



Подробно форматы рассматриваются в следующих разделах.

3.7.1. Журнал экспорта

Операции экспорта-импорта формируют журнал с подробным отчётом о выполненных действиях. Журнал представляет собой простой (plain) текстовый файл и получить его можно вызовом эндпоинта `GET /zenith-object/api/v1/log/export`. В запросе можно указать необязательный параметр `delete` - признак того, нужно ли удалять файл журнала на сервере Зенита (значения `true/false`).

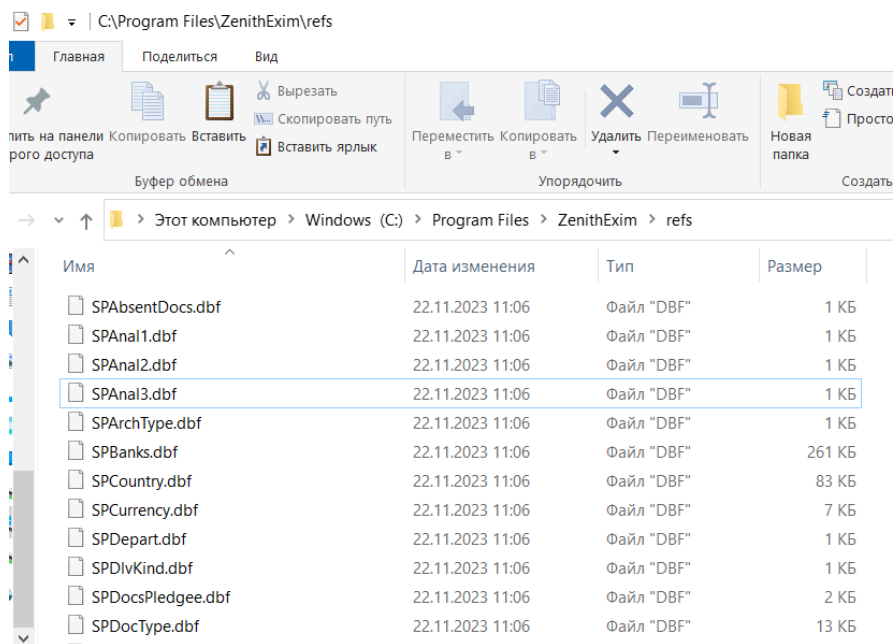
Пример запроса:

```
curl -X 'GET' \  
  'http://localhost:8080/zenith-object/api/v1/log/export?delete=false' \  
  -H 'accept: text/plain' \  
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \  
  -H 'X-Zenith-Server: C0'
```

Журнал всегда относится к последней операции и его удобно использовать для разбора ошибок.

3.7.2. Экспорт справочников

В отличие от остальных форматов, шаблон для справочников в модуле экспорта-импорта нельзя сохранить в произвольный каталог. Вместо этого нужно открыть каталог установки модуля (обычно это `C:\Program Files\ZenithExIm`) и там, в подкаталоге `refs`, вы найдёте данные справочников.



Содержимое каталога лучше скопировать в какую-либо свою папку и использовать в качестве шаблона справочников.

Для экспорта справочников используйте вызов эндпоинта `POST /zenith-object/api/v1/export/refs`. В качестве параметра необходимо в переменной `subsystem` указать подсистему, экспорт справочников которой осуществляется (значения `svr` для подсистемы СВР или `pif` для подсистемы ПИФ), а также в теле запроса передать zip-архив со структурой справочников.

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/export/refs?subsystem=svr' \
  -H 'accept: application/zip' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/zip' \
  --data-binary '@refs.zip'
```

В ответ сервер присылает архив с набором DBF-файлов, заполненных данными справочников.

3.7.3. Экспорт формата состояний

Для экспорта реестра в формате состояний необходимо использовать вызов эндпоинта `POST /zenith-object/api/v1/issuers/{issuer}/export/state`, в параметре запроса указывается код эмитента, реестр которого выгружается.

В теле запроса необходимо передать следующие параметры:

- `templateZip` - архив с файлами шаблона формата состояний,
- `stateFilter` - описание фильтра выгрузки (необязательный параметр),
- `stateFilterXmlPart` - XML-файл с параметрами фильтрации счетов (необязательный параметр).

Описание фильтра выгрузки передается в виде `json` следующего формата (обязательные параметры помечены *):

- `exportDate` - дата, состояние реестра на которую нужно выгрузить. Если не задана, выгружается текущее состояние;
- `currentForm` - выгружать текущие анкетные данные (а не на дату `exportDate`), значения `true/false`. Параметр важен только при выгрузке на заданную дату. Значение `false` (по умолчанию) означает, что анкеты выгружаемых ЗЛ будут браться на дату выгрузки, `true` требует брать текущее состояние анкет;
- `operationsToTheEnd` - выгружать журнал операций до конца (а не до `exportDate`), значения `true/false`. Параметр важен только при выгрузке на заданную дату и только если выгружается журнал операций (таблица `Charges.dbf`). Значение `false` (по умолчанию) означает, что выгружаются только операции до даты `exportDate` включительно, `true` выгружает полный журнал операций;
- `includeND` - включать ли раскрытия номинальных держателей на дату выгрузки `exportDate`, значения `true/false`. Параметр важен только при выгрузке на заданную дату. Значение `true` требует экспортировать так же и раскрытия номинальных держателей, если они имеются на указанную дату. Значение `false` (по умолчанию) означает, что раскрытия номинальников не включаются. Раскрытия добавляются в конец таблицы `Account.dbf`. Есть важная особенность: параметр используется, только если в таблице `Account.dbf` присутствует поле `ACCOUNT_ND`, содержащее номер счёта внутри раскрытия. Рекомендуем в модуле экспорта-импорта использовать типовой шаблон "Базовый для собраний". В шаблоне "Базовый для состояний" данное поле отсутствует;
- `imageFilter` - параметры выгрузки изображений (скан-образов). При отсутствии `imageFilter` изображения не выгружаются. Фильтр представляет собой объект

следующего формата:

- `onlyActive*` - выгружать только действующие, значения `true/false`;
- `typeFilter*` - по каким ЗЛ выгружать изображения:
 - `{"type": "all"}` - по всем,
 - `{"type": "individual"}` - по ФЛ,
 - `{"type": "legal"}` - по ЮЛ;
- `outDir*` - папка для выгружаемых данных, должен быть открыт общий доступ с правами на запись;
- `generateUnstructAddress` - генерировать поля неструктурированного адреса на основе структурированного, значения `true/false`. Данная опция позволяет не разбираться, структурированный или неструктурированный адрес задан в реестре, а всегда брать готовый текст из неструктурированного поля. Удобно использовать, например, если реестр выгружается для подготовки списка к собранию;
- `nonEmptyAccountsOnly` - выгружать только счета, на которых имеются ЦБ, значения `true/false`. При выгрузке всех счетов этот параметр игнорируется;
- `alwaysIncludeNDCD` - счет НД ЦД выгружать даже если на нем нет ЦБ, значения `true/false`;
- `corpactMeetingId` - номер КД типа "Собрание";
- `corpactDisclosureId` - номер КД типа "Раскрытие информации";
- `accountRange` - выгружать только счета из заданного диапазона. Объект с полями:
 - `from*` - "от",
 - `to*` - "до";
- `filterByState` - фильтровать по состоянию счёта:
 - `{"type": "archivedAccount"}` - счет, переданный в архив,
 - `{"type": "canceledAccount"}` - аннулированный счет,
 - `{"type": "validAccount"}` - действующий счет.
- `accountsOpenRange` - фильтровать по операционному дню открытия счёта. Объект с полями:
 - `from*` - "от",
 - `to*` - "до";
- `accountsCloseRange` - фильтровать по операционному дню закрытия счёта. Объект с полями:
 - `from*` - "от",
 - `to*` - "до";
- `accountsLastOpRange` - фильтровать по операционному дню последней операции над счётом. Объект с полями:
 - `from*` - "от",
 - `to*` - "до";

Если фильтр не указан, то будет выгружено текущее состояние реестра одного эмитента, без фильтрации данных (в частности, будут включены нулевые счета).

Параметры `exportDate`, `compactMeetingId` и `compactDisclosureId` взаимоисключающие — заполнять можно не более одного из них. `compactMeetingId` и `compactDisclosureId` разными способами определяют, на какую дату будут взяты состояния счетов (то есть косвенно задают `exportDate`) и дополнительно определяют, данные какого раскрытия будут выгружены (раскрытия привязаны к конкретным КД).

Изображения из карточки эмитента выгружаются всегда, вне зависимости от значения `imageFilter.typeFilter`. Изображения помещаются в подкаталог `img`, именование и содержимое файлов точно такое же, как при выгрузке из модуля экспорта-импорта, см. `ZenithExim.doc`.

Пример запроса с `json`-фильтром:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/issuers/5/export/state' \
  -H 'accept: application/zip' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: multipart/form-data' \
  -F 'templateZip=@dbf-sost.zip;type=application/x-zip-compressed' \
  -F 'stateFilter={
    "exportDate": "2024-11-08",
    "currentForm": true,
    "operationsToTheEnd": true,
    "includeND": true,
    "generateUnstructAddress": true,
    "nonEmptyAccountsOnly": true,
    "alwaysIncludeNDCD": true,
    "accountRange": {
      "from": 2,
      "to": 10000
    },
    "accountsOpenRange": {
      "from": "2024-11-08",
      "to": "2024-11-08"
    },
    "accountsCloseRange": {
      "from": "2024-11-08",
      "to": "2024-11-08"
    },
    "accountsLastOpRange": {
      "from": "2024-11-08",
      "to": "2024-11-08"
    }
  }
```

```
}  
}'
```

Самую подробную фильтрацию счетов можно организовать через фильтр в XML-формате, который можно передать в теле запроса. Схема данных фильтра описана в файле `zenith_filters.xsd`, элемент `FILTERZL`. В экспорте поддерживаются не все поля, теоретически доступные для фильтра, а только те, что можно увидеть в модуле экспорта-импорта, на бланке фильтрации. Для примера сохраним в файл `accard-filter.xml` следующий фильтр:

```
<?xml version="1.0" encoding="windows-1251"?>  
<FILTERZL>  
  <COMMON>  
    <ACCTYPE>  
      <INC>1</INC>  
      <LIST>4;3</LIST> <!-- только владельцы и номинальщики -->  
    </ACCTYPE>  
    <ACCARCH> <!-- только действующие счета -->  
      <LIST>0</LIST>  
      <INC>1</INC>  
    </ACCARCH>  
    <CITIZEN> <!-- только тех, у кого гражданство - Украина -->  
      <INC>1</INC>  
      <TYPE>2</TYPE>  
      <LIST>126</LIST>  
    </CITIZEN>  
  </COMMON>  
</FILTERZL>
```

Ему соответствует следующий бланк в модуле экспорта-импорта:

Пример запроса с XML-фильтром:

```
'http://localhost:8080/zenith-object/api/v1/issuers/1/export/state' \
-H 'accept: application/zip' \
-H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
-H 'X-Zenith-Server: C0' \
-H 'Content-Type: multipart/form-data' \
-F 'templateZip=@dbf-sost.zip;type=application/x-zip-compressed' \
-F 'stateFilterXmlPart=@accard_filter.xml;type=text/xml'
```

В ответ сервер присылает архив с выгрузкой реестра по состоянию, счета в выгрузке отфильтрованы согласно переданным параметрам.

3.7.4. Экспорт исторического формата

Для экспорта реестра в историческом формате необходимо использовать вызов эндпоинта `POST /zenith-object/api/v1/issuers/{issuer}/export/history` в параметре запроса указывается код эмитента, реестр которого выгружается.

В теле запроса необходимо передать следующие параметры:

- `templateZip` - архив с файлами шаблона формата состояний,
- `historyFilter` - описание фильтра выгрузки (необязательный параметр),
- `historyFilterXmlPart` - XML-параметры фильтрации счетов (необязательный параметр).

Описание фильтра выгрузки передается в виде `json` следующего формата (обязательные параметры помечены *):

- `dateFilter` - данные выгружаемого периода, представляет собой объект со следующими полями:
 - `startDate*` - начало выгружаемого периода истории,
 - `endDate*` - конец выгружаемого периода истории,
 - `includeStateOnEndDate*` - включать ли в выгрузку состояние на конец периода, значения `true/false`;
- `includeCurrentState` - дополнительно выгрузить состояние реестра на текущий момент (при выгрузке полной истории), значения `true/false`;
- `onlyReg` - способ отбора журналов, значения `true/false`. Значение `false` (по умолчанию) означает, что из журналов документов, исходящих и операций будут взяты фрагменты за период, заданный через `startDate/endDate`. Значение `true` включает несколько иное поведение: на основе `startDate/endDate` будет взят только фрагмент журнала операций, а из журналов входящих и исходящих будут выгружены те строки, которые относятся к выбранным операциям;

- `imageFilter` - параметры загрузки изображений (скан-образов). Объект содержит следующие поля:
 - `docImages` - загрузить скан-образы документов, приложений и исходящих, значения `true/false`,
 - `accountImages` - загрузить скан-образы, прикрепленные к карточкам ЗЛ (анкеты и другие), значения `true/false`,
 - `manualOutDocFiles` - загрузить файлы ручных исходящих, значения `true/false`,
 - `edoFiles` - загрузить файлы ЭДО-сообщений входящих и исходящих документов, значения `true/false`,
 - `outDir*` - папка для загружаемых данных, должен быть открыт общий доступ с правами на запись.

Параметры `imageFilter.docImages` и `imageFilter.accountImages` включают загрузку скан-образов, прикрепленных к соответствующим объектам. Файлы загружаются в подкаталог `img`, схема именования та же, что при загрузке из модуля экспорта-импорта, см. `ZenithExim.doc`. Сканы документов начинаются с префикса `ID_`, изображения из карточек ЗЛ — с префикса `ACC_`.

Сканы из карточек ЗЛ могут быть загружены, только если в дополнение к истории загружается состояние счетов. При загрузке фрагмента истории это включается последним параметром (`dateFilter.includeStateOnEndDate=true`). При загрузке полной истории, используйте отдельное свойство `includeCurrentState`.

Установленное в `true` свойство `imageFilter.manualOutDocFiles` загружает в подкаталог `outdoc` файлы всех ручных исходящих, участвующих в экспорте.

Установленное `true` свойство `imageFilter.edoFiles` загружает в подкаталог `edo` ЭДО-файлы всех экспортируемых входящих и исходящих документов. Файлы, начинающиеся с `ID_` относятся ко входящим, с `OD_` — к исходящим.

Пример запроса с `json`-фильтром:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/issuers/5/export/history' \
  -H 'accept: application/zip' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: multipart/form-data' \
  -F 'templateZip=@dbf-hist.zip;type=application/x-zip-compressed' \
  -F 'historyFilter={
    "dateFilter": {
      "startDate": "2020-01-15",
      "endDate": "2024-11-10",
      "includeStateOnEndDate": true
    }
  }
```

```
},  
  "onlyReg": true  
}'
```

Исторический формат также допускает дополнительную фильтрацию, задаваемую XML-документом, который можно передать в теле запроса. Фильтрации подвергается перечень операций технологического журнала.

Формальное описание схемы XML-фильтра находится в файле `zenith_filters.xsd`, в элементе `FILTERRG`. При экспорте используется не все поля, а только те, что определены в бланке фильтрации модуля экспорта-импорта. Пример файла с фильтром:

```
<?xml version="1.0" encoding="windows-1251" ?>  
<FILTERRG>  
  <COMMON>  
    <OPERNUM_TYPE>2</OPERNUM_TYPE>  
    <CHTYPE>  
      <INC>1</INC>  
      <LIST>60101;60102;60103;60104;60105;60106;60107;60108</LIST>  
    </CHTYPE>  
    <OPERSTAT>  
      <INC>1</INC>  
      <VALUE>0</VALUE>  
    </OPERSTAT>  
  </COMMON>  
</FILTERRG>
```

Данный фильтр оставляет только исполненные (не отвергнутые) операции с типами 60101-60108 (различные виды трансфертов). В модуле экспорта-импорта ему соответствует следующий бланк:

Фильтр журнала поручений

Операции

- ☒ 60101-Переход прав собственности при совершении...
- ☒ 60102-Переход прав собственности в результате дар...
- ☒ 60103-Переход прав собственности в результате нас...
- ☒ 60104-Переход прав собственности при реорганизаци...
- ☒ 60105-Переход прав собственности по решению суда
- ☒ 60106-Переход прав собственности по договору мен...
- ☒ 60107-Внесение ценных бумаг в уставный капитал
- ☒ 60108-Передача ЦБ при невыполнении условий залог
- ☐ 60109-Передача ЦБ при реорганизации путем присое...
- ☐ 60110-Передача неоплаченных ценных бумаг
- ☐ 60111-Передача заложенных ЦБ
- ☐ 60112-Переход прав собственности при выкупе по тр...
- ☐ 60120-Передача ЦБ по иным основаниям
- ☐ 60121-Передача ЦБ при объединении лицевых счетов
- ☐ 60170-Передача ЦБ вследствие отмены операции
- ☐ 60201-Первичное размещение
- ☐ 60202-Размещение ЦБ при приеме реестра по состо...
- ☐ 60203-Зачисление ЦБ при приеме реестра по состо...
- ☐ 60204-Размещение ЦБ при реорганизации АО
- ☐ 60205-Конвертация ЦБ при реорганизации АО
- ☐ 60220-Зачисление ЦБ
- ☐ 60221-Зачисление ЦБ при проведении глобальной оп...
- ☐ 60230-Размещение дополнительных ЦБ среди владе...
- ☐ 60231-Размещение ЦБ при проведении капитализаци...
- ☐ 60240-Зачисление ЦБ на эмиссионный счет эмитентом
- ☐ 60301-Выкуп(приобретение) ценных бумаг эмитентом
- ☐ 60302-Приобретение ЦБ эмитентом для погашения
- ☐ 60303-Возврат при несостоявшейся эмиссии
- ☐ 60304-Списание ЦБ при реорганизации/ликвидации

Технологический журнал по

Регистрационный журнал по

☒ Отмеченные

☐ Только валютные операции

Номер лицевого счета по

По ценным бумагам

Вид Количество ЦБ с по

Кат. Сумма сделки с по

Тип

☒ Статус поручения

☒ Исполнено

☐ Отвергнуто

Исполнитель 100-Администратор

сохранить загрузить Скрыть...

Пример запроса с XML-фильтром:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/issuers/1/export/history' \
  -H 'accept: application/zip' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: multipart/form-data' \
  -F 'templateZip=@dbf-hist.zip;type=application/x-zip-compressed' \
  -F 'historyFilterXmlPart=@filter.xml;type=text/xml'
```

В ответ сервер присылает архив с выгрузкой реестра в историческом формате, в котором журнал операций отфильтрован согласно установленному фильтру.

3.7.5. Экспорт документа на оплату в бухгалтерию

Вызов эндпоинта `POST /zenith-object/api/v1/documents/{doc_id}/bill` позволяет создать и экспортировать документ на оплату. В запросе указывается два обязательных параметра:

- `doc_id` - идентификатор документа, чьи услуги нужно внести в создаваемый документ на оплату,
- `bill_type` - тип документа на оплату.

В документ на оплату включаются только те услуги, которые ещё не включены в другие документы на оплату.

Пример запроса на экспорт документа на оплату:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/documents/607/bill?bill_type=1' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -d ''
```

3.8. Импорт данных

3.8.1. Импорт котировок ценных бумаг

Московская биржа предоставляет подписчикам информацию о котировках ценных бумаг в специальном файле формата XML. Котировки из этого файла можно загружать через Зенит-API. Для этого необходимо вызвать эндпоинт `PUT /zenith-object/api/v1/import/securities_quotes`, в запросе указать обязательный параметр `operday` - операционный день, по которому загружаются котировки, а в теле запроса необходимо передать котировки ММВБ в формате xml.

В файле могут быть котировки и за другие дни, они игнорируются. Однако отсутствие котировок за заданный день считается ошибкой, означающей, что, вероятно, загружается не тот файл.

Котировки загружаются только по ЦБ тех реестров, которые ведутся в текущем подразделении, котировки всех прочих ЦБ игнорируются.

Если у встреченной в файле ценной бумаги не установлен признак котируемости, он устанавливается автоматически.

Сервер возвращает количество ЦБ, по которым загружены котировки. Отметим, что выбирать текущий операционный день для загрузки котировок не требуется. Так же не требуется, чтобы день, по которому загружаются котировки был открыт, он может быть закрытым. Загруженные котировки можно посмотреть в Зенит-клиенте в режиме "Действия с операционными днями", кнопка "Котировки ценных бумаг".

Пример запроса:

```
curl -X 'PUT' \
  '<http://localhost:8080/zenith-object/api/v1/import/securities_quotes'\
  '?operday=2024-11-11>' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/xml' \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<document>
```



```
<data id="history">
<metadata>
<columns>
<column max_size="0" bytes="10" type="date" name="TRADEDATE"/>
<column max_size="0" bytes="36" type="string" name="SECID"/>
<column max_size="0" bytes="381" type="string" name="BOARDNAME"/>
<column max_size="0" bytes="12" type="string" name="BOARDID"/>
<column max_size="0" bytes="189" type="string" name="SHORTNAME"/>
<column max_size="0" bytes="189" type="string" name="REGNUMBER"/>
<column max_size="0" bytes="765" type="string" name="ISIN"/>
<column max_size="0" bytes="6" type="string" name="LISTNAME"/>
<column type="double" name="FACEVALUE"/>
<column max_size="0" bytes="9" type="string" name="CURRENCYID"/>
<column type="double" name="PREVLEGALCLOSEPRICE"/>
<column type="double" name="PREV"/>
<column type="double" name="LEGALOPENPRICE"/>
<column type="double" name="OPENPERIOD"/>
<column type="double" name="OPEN"/>
<column type="double" name="LOW"/>
<column type="double" name="HIGH"/>
<column type="double" name="CLOSE"/>
<column type="double" name="CLOSEPERIOD"/>
<column type="double" name="OPENVAL"/>
<column type="double" name="CLOSEVAL"/>
<column type="double" name="LEGALCLOSEPRICE"/>
<column type="double" name="TRENDCLOSE"/>
<column type="double" name="TRENDCLSPR"/>
<column type="double" name="HIGHBID"/>
<column type="double" name="BID"/>
<column type="double" name="OFFER"/>
<column type="double" name="LOWOFFER"/>
<column type="double" name="WAPRICE"/>
<column type="double" name="TRENDWAP"/>
<column type="double" name="TRENDWAPPR"/>
<column type="double" name="VOLUME"/>
<column type="double" name="MARKETPRICE"/>
<column type="double" name="MARKETPRICE2"/>
<column type="double" name="MP2VALTRD"/>
<column type="double" name="MPVALTRD"/>
<column type="double" name="VALUE"/>
<column type="double" name="NUMTRADES"/>
<column type="double" name="ISSUESIZE"/>
<column type="double" name="ADMITTEDQUOTE"/>
<column type="double" name="ADMITTEDVALUE"/>
<column type="double" name="MONTHLYCAPITALIZATION"/>
<column type="double" name="DAILYCAPITALIZATION"/>
<column type="double" name="MARKETPRICE3"/>
<column type="double" name="MARKETPRICE3TRADESVALUE"/>
<column type="int32" name="DECIMALS"/>
<column max_size="0" bytes="93" type="string" name="TYPE"/>
<column type="double" name="CLOSEAUCTIONPRICE"/>
```

```

<column type="double" name="WAVAL"/>
<column type="double" name="LASTPRICE"/>
<column type="double" name="MARKETPRICE3TRADESVALUECUR"/>
<column type="double" name="MARKETPRICE3CUR"/>
<column type="int32" name="TRADINGSESSION"/>
</columns>
</metadata>
<rows>
<row CURRENCYID="RUR" DECIMALS="1" VALUE="298870" LOW="129.5" HIGH="130.5" OPEN="129.5" CL
<row CURRENCYID="RUR" DECIMALS="2" VALUE="437892" LOW="4.9" HIGH="5.14" OPEN="5.08" CL
</rows>
</data>
<data id="history.cursor">
<metadata>
<columns>
<column type="int64" name="INDEX"/>
<column type="int64" name="TOTAL"/>
<column type="int64" name="PAGESIZE"/>
</columns>
</metadata>
<rows>
<row PAGESIZE="100" TOTAL="491" INDEX="0"/>
</rows>
</data>
</document>'

```

3.9. Вспомогательные запросы

3.9.1. Получение идентификатора текущего подразделения

Для получения идентификатора текущего подразделения используйте вызов эндпоинта `GET /zenith-object/api/v1/subdivision/id`.

Пример запроса:

```

curl -X 'GET' \
  'http://localhost:8080/zenith-object/api/v1/subdivision/id' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'

```

В ответ сервер присылает идентификатор текущего подразделения в виде числового значения.

3.9.2. Получение количества КД типа "Раскрытие НД" на заданную дату

Для того, чтобы получить количество КД типа "Раскрытие НД" на заданную дату, используется вызов эндпоинта `POST /zenith-object/api/v1/corporate_actions/nominees_disclosures/count`. В теле запроса передается объект с данными КД в виде `json` со следующими полями:

- `issuer` - код эмитента,
- `disclosureDate` - дата раскрытия.

Пример такого объекта:

```
{
  "issuer": 5,
  "disclosureDate": "2024-10-30"
}
```

Пример запроса:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/corporate_actions/' \
  'nominees_disclosures/count' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/json' \
  -d '{
    "issuer": 5,
    "disclosureDate": "2024-10-30"
  }'
```

В ответ сервер присылает количество найденных КД раскрытия на указанную дату в виде числа.

4. Прямые запросы к Зенит-Серверу

Существует возможность отправлять произвольные запросы к Зенит-серверу в том же самом формате, в каком их выполняет Зенит-клиент.

Работа происходит следующим образом - необходимо подготовить запрос и вызвать эндпоинт `POST /zenith-object/api/v1/raw`, при вызове передать подготовленный запрос в теле запроса в виде XML.

Пример прямого запроса к серверу Зенита, осуществляющий выбор операционного дня:

```
curl -X 'POST' \
  'http://localhost:8080/zenith-object/api/v1/raw' \
  -H 'accept: application/xml' \
```

```
-H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \  
-H 'X-Zenith-Server: C0' \  
-H 'Content-Type: application/xml' \  
-d '<?xml version="1.0" encoding="windows-1251"?>  
<__MESSAGE METHOD_NAME="CHANGEMYOPERDAY" OBJ_NAME="ZENITH" OBJ_ID="0" SOURCE="INTF">  
  <ODLIST>  
    <OPERDAY>11.11.2024</OPERDAY>  
  </ODLIST>  
</__MESSAGE>  
'
```

В ответ на запрос в случае успешного выбора операционного сервер присылает ответ об успешном исполнении запроса:

```
<__MESSAGE>  
  <__RESULT>  
    <CODE>0</CODE>  
  </__RESULT>  
</__MESSAGE>
```

Эта документация описывает отдельные запросы, которые можно выполнять. Полной пользовательской документации по всем существующим в Зените запросам нет, она готовится под конкретные потребности клиентов.

4.1. Структура запроса и ответа

И запрос и ответ представляют собой XML-документы в кодировке `windows-1251`, представляемые в виде простой, закрытой нулём строки.

Пробелы и табуляции между элементами роли не играют и не анализируются.

Корневой элемент сообщения всегда `<__MESSAGE>`. В запросе там задаются основные атрибуты запроса: что делаем (`METHOD_NAME`), над каким объектом делаем (`OBJ_NAME` и `OBJ_ID`). Атрибут `SOURCE` определяет источник запроса, здесь необходимо указать `"INTF"` — пользовательский интерфейс.

В ответе элемент `<__MESSAGE>` атрибутов не имеет. Однако в ответе у `<__MESSAGE>` всегда есть дочерний элемент `<__RESULT>`, описывающий итог выполнения запроса. Внутри `<__RESULT>` элемент `<CODE>` содержит целое число, определяющее успешность: `0` — успешно, не ноль — проблемы, их в виде текстового описания передает элемент `<DESCRIPTION>`.

Все данные предметной области передаются в элементах, в атрибутах лежат только различные служебные данные.

XML-элементы в рамках родителя делятся на одиночные и списковые.

Порядок следования одиночных элементов не важен, на него не полагается Зенит-сервер и не должен полагаться клиент. Обращение к элементу всегда идёт по имени.

Списковый элемент всегда имеет атрибут `ID`, однозначно идентифицирующий элемент внутри списка. Зенит-клиент обычно (но необязательно) отображает строки в физическом порядке следования элементов.

В запросах на чтение задаётся шаблон читаемых данных: перечисляются все элементы, которые вам нужны (пустые, без данных), сервер запрошенное возвращает. Сервер должен вернуть всё, что попросили (если такие элементы существуют в принципе) и имеет право вернуть больше. Имена элементов не проверяются на корректность, ошиблись в имени — сервер игнорирует такой элемент.

При чтении списка внутри шаблона чтения может задаваться элемент `<FILTER>`, по содержимому которого осуществляется отбор списковых элементов. Для каждого вида списков фильтр имеет свой формат.

4.2. Выбор операционного дня

Перед выполнением некоторых запросов требуется выбрать операционный день, под которыми они выполняются. Такие запросы, если день не выбран, возвращают ошибку с кодом `1001001`.

Выбор операционного дня осуществляется специальным запросом:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE METHOD_NAME="CHANGEMYOPERDAY" OBJ_NAME="ZENITH" OBJ_ID="0" SOURCE="INTF">
  <ODLIST>
    <OPERDAY>11.11.2024</OPERDAY>
  </ODLIST>
</__MESSAGE>
```

Дату нужно подавать в формате `ДД.ММ.ГГГГ`.

Выбирать можно только операционный день из числа открытых. Выбранный день ассоциируется с соединением, в рамках которого он выбран и сохраняется до разрыва подключения с Зенит-сервером.

Так же важно помнить, что другой пользователь может в любой момент закрыть операционный день, и другие пользователи, у кого он был выбран, своего выбора лишаются. Даже если этот день будет потом открыт повторно, автоматически он не выберется — рекомендуется реагировать на ошибку `1001001` и повторно слать запрос на выбор дня.

4.3. Получение данных справочника

4.3.0.1. Чтение по фильтру

Пример XML-запроса к серверу Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<!-- Запрос на чтение справочника: _REF.READ -->
<__MESSAGE METHOD_NAME="READ" OBJ_NAME="_REF" OBJ_ID="0" SOURCE="INTF">
  <_REF>
    <!-- ОПФ: имя справочника организационно-правовых форм -->
    <OPF>
      <!-- DATA: внутри шаблон читаемых данных -->
      <DATA>
        <!-- фильтр задаёт, что читаем -->
        <FILTER>
          <!-- простые (не групповые) строки -->
          <_GROUP>0</_GROUP>
          <!-- код родительской группы -->
          <_PID>39</_PID>
          <!-- уровень вложенности -->
          <_LEVEL>1</_LEVEL>
        </FILTER>
        <!-- список читаемых полей: -->
        <_DEL/> <!-- признак логического удаления -->
        <_FIX/> <!-- признак неизменной строки -->
        <NAME/> <!-- наименование ОПФ -->
        <CODE/> <!-- аббревиатура ОПФ -->
      </DATA>
    </OPF>
  </_REF>
</__MESSAGE>
```

Пример ответа сервера:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
    <OPF>
      <!-- список запрошенных строк -->
      <DATA ID="68">
        <!-- полей может быть возвращено больше, чем запрошено,
              это нормально -->
        <CODE>ДУП</CODE>
        <NAME>Дочернее унитарное предприятие</NAME>
        <_DEL>0</_DEL>
        <_FIX>1</_FIX>
        <_GROUP>0</_GROUP>
        <_LEVEL>1</_LEVEL>
        <_MARK>0</_MARK>
        <_PID>39</_PID>
```

```

</DATA>
<DATA ID="67">
  <CODE>3A0</CODE>
  <NAME>Закрытое акционерное общество</NAME>
  <_DEL>0</_DEL>
  <_FIX>1</_FIX>
  <_GROUP>0</_GROUP>
  <_LEVEL>1</_LEVEL>
  <_MARK>0</_MARK>
  <_PID>39</_PID>
</DATA>
<DATA ID="47">
  <CODE>0A0</CODE>
  <NAME>Открытое акционерное общество</NAME>
  <_DEL>0</_DEL>
  <_FIX>1</_FIX>
  <_GROUP>0</_GROUP>
  <_LEVEL>1</_LEVEL>
  <_MARK>0</_MARK>
  <_PID>39</_PID>
</DATA>
<DATA ID="51">
  <CODE>ПТ</CODE>
  <NAME>Полное товарищество</NAME>
  <_DEL>0</_DEL>
  <_FIX>1</_FIX>
  <_GROUP>0</_GROUP>
  <_LEVEL>1</_LEVEL>
  <_MARK>0</_MARK>
  <_PID>39</_PID>
</DATA>
</OPF>
</_REF>

<!-- успешность выполнения запроса -->
<__RESULT>
  <!-- 0: порядок, строки возвращены -->
  <CODE>0</CODE>
  <!-- если не 0, тут был бы ещё элемент DESCRIPTION -->
</__RESULT>
</__MESSAGE>

```

4.3.0.2. Чтение по идентификатору

Чтение по идентификатору аналогично чтению по фильтру, только вместо фильтра задаётся идентификатор читаемой строки.

Пример XML-запроса к серверу Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE METHOD_NAME="READ" OBJ_NAME="_REF" OBJ_ID="0" SOURCE="INTF">
  <_REF>
    <OPF>
      <DATA ID="67">
        <_DEL/>
        <_FIX/>
        <NAME/>
        <CODE/>
      </DATA>
    </OPF>
  </_REF>
</__MESSAGE>
```

Пример ответа сервера Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
    <OPF>
      <DATA ID="67">
        <CODE>3A0</CODE>
        <NAME>Закрытое акционерное общество</NAME>
        <_DEL>0</_DEL>
        <_FIX>1</_FIX>
        <_GROUP>0</_GROUP>
        <_LEVEL>1</_LEVEL>
        <_MARK>0</_MARK>
        <_PID>39</_PID>
      </DATA>
    </OPF>
  </_REF>
  <__RESULT>
    <CODE>0</CODE>
  </__RESULT>
</__MESSAGE>
```

4.4. Получение текущей версии и сервис-пака

Номер тех.версии и сервис-пака хранится в полях справочника "Территориальная инфраструктура" (внутреннее имя `SUBDIV`), соответственно, их чтение — обычное чтение строки справочника.

Код строки, описывающей центральный офис всегда равен 1.

Пример XML-запроса к серверу Зенита


```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE METHOD_NAME="READ" OBJ_NAME="_REF" OBJ_ID="0" SOURCE="INTF">
  <_REF>
    <SUBDIV>
      <DATA ID="1">
        <TECHVER/> <!-- поле номера тех.версии -->
        <SP/> <!-- поле номера сервис-пака -->
      </DATA>
    </SUBDIV>
  </_REF>
</__MESSAGE>
```

Пример ответа сервера:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
    <SUBDIV>
      <DATA ID="1">
        <TECHVER>28</TECHVER>
        <SP>2</SP>
      </DATA>
    </SUBDIV>
  </_REF>
  <__RESULT>
    <CODE>0</CODE>
  </__RESULT>
</__MESSAGE>
```

4.5. Чтение данных справочника, изменившихся с контрольной точки

Все изменения в строках справочника отражаются в специальном журнале. Записи в журнале пронумерованы целыми числами по возрастанию, начиная с **1**, эти числа называются контрольными точками. Если вы храните у себя копию справочников Зенита, вы должны вместе со справочником хранить и номер контрольной точки до которой вы обновились.

В запросе на обновления вы указываете вашу контрольную точку, Зенит возвращает новую и (если вы запросили) изменённые/добавленные строки. При первом запросе в качестве контрольной точки передавайте **0**.

Пример XML-запроса к серверу Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<!-- _REF.READ_REF_UPDATES – метод чтения обновлений справочника -->
```

```

<__MESSAGE METHOD_NAME="READ_REF_UPDATES" OBJ_NAME="_REF" OBJ_ID="0"
  SOURCE="INTF">
  <_REF>
  <OPF>
    <!-- ваша контрольная точка -->
    <CHECK_POINT>7</CHECK_POINT>

    <!-- если вам нужны и обновлённые строки, указывайте элемент
          DATA -->
    <DATA>
      <!-- тут перечисляйте нужные поля, если их нет, будут
            прочитаны все имеющиеся -->
      <NAME/>
      <CODE/>
    </DATA>
  </OPF>
  <!-- тут можете запросить данные других справочников аналогичным
        образом -->
</_REF>
</__MESSAGE>

```

Пример ответа сервера, если новых данных нет:

```

<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
  <OPF>
    <!-- даже если данных нет, КТ может измениться -->
    <CHECK_POINT>10</CHECK_POINT>
  </OPF>
</_REF>
<__RESULT>
  <CODE>0</CODE>
</__RESULT>
</__MESSAGE>

```

Пример ответа сервера Зенита при наличии новых данных:

```

<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
  <OPF>
    <CHECK_POINT>11</CHECK_POINT>
    <DATA ID="67">
      <CODE>3A0</CODE>
      <NAME>Закрытое акционерное общество</NAME>
    </DATA>
  </OPF>
</_REF>

```

```
<__RESULT>
  <CODE>0</CODE>
</__RESULT>
</__MESSAGE>
```

После параметра `CHECK_POINT` можете добавить необязательный параметр `MAX_ROW_COUNT`, задающий максимальное число строк `<DATA>`, которые хотите получить. Задавать его имеет смысл, если вы допускаете, что в справочнике с прошлой контрольной точки могут измениться много (тысячи) строк и предпочитаете обрабатывать их по частям. В этом случае пишете в `MAX_ROW_COUNT` некоторое число строк, чтобы ограничить размер приходящей порции.

4.6. Добавление строки в справочник

Пример XML-запроса к серверу Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<!-- REFUSER.ADD означает добавление строки в пользовательский справочник -->
<__MESSAGE METHOD_NAME="ADD" OBJ_NAME="REFUSER" OBJ_ID="0" SOURCE="INTF">
  <_REF>
    <!-- OPF: имя справочника организационно-правовых форм -->
    <OPF>
      <!-- операция INSERT отмечает добавляемую строку -->
      <DATA OPERATION="INSERT">
        <NAME>Новое аномально научное общество</NAME>
        <CODE>НАНО</CODE>
        <_GROUP>0</_GROUP>
        <_PID>39</_PID>
        <_LEVEL>1</_LEVEL>
      </DATA>
      <!-- здесь может быть следующая добавляемая строка -->
    </OPF>
  </_REF>
</__MESSAGE>
```

Если всё прошло успешно, ответ возвращает идентификаторы добавленных строк (одна в нашем примере):

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <_REF>
    <OPF>
      <DATA ID="76"/>
    </OPF>
  </_REF>
<__RESULT>
  <CODE>0</CODE>
```

```
</__RESULT>
</__MESSAGE>
```

4.7. Изменение строки справочника

Пример XML-запроса к серверу Зенита:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE METHOD_NAME="CHANGE" OBJ_NAME="REFUSER" OBJ_ID="0" SOURCE="INTF">
  <_REF>
    <!-- OPF: внутреннее имя справочника организационно-правовых
           форм -->
    <OPF>
      <!-- операция UPDATE задаёт факт, что строка меняется, ID
           определяет, какая именно -->
      <DATA ID="76" OPERATION="UPDATE">
        <!-- тут задаются значения только изменяемых полей -->
        <NAME>Научно-атлетическое нано-общество</NAME>
      </DATA>
    </OPF>
  </_REF>
</__MESSAGE>
```

Пример ответа сервера, который просто подтверждает успешность выполнения:

```
<?xml version="1.0" encoding="windows-1251"?>
<__MESSAGE>
  <__RESULT>
    <CODE>0</CODE>
  </__RESULT>
</__MESSAGE>
```

4.8. Загрузка файла на сервер

Для загрузки файла на сервер при использовании прямых запросов к Зениту используется отдельный эндпоинт `POST /zenith-object/api/v1/raw/files/upload`. В теле запроса передается загружаемый файл. Например:

```
curl -X 'POST' \
  'http://localhost:18080/zenith-object/api/v1/raw/files/upload' \
  -H 'accept: text/plain' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0' \
  -H 'Content-Type: application/octet-stream' \
  --data-binary '@consolidated.xml'
```

Если файл загружен успешно, возвращается код 200 и имя файла в формате Зенита, например \4d334c8b-5d05-4781-b5f7-d831903535b6

4.9. Загрузка файла с сервера

Для загрузки файла с сервера при использовании прямых запросов к Зениту используется отдельный эндпоинт GET /zenith-object/api/v1/raw/files/download. В параметре запроса filename передается имя файла в формате Зенита. Например:

```
curl -X 'GET' \
  'http://localhost:18080/zenith-object/api/v1/raw/files/download?filename=%5C4d334c8b-5d05-4781-b5f7-d831903535b6' \
  -H 'accept: application/octet-stream' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В случае успешного ответа возвращается код 200 и файл в теле ответа.

4.10. Удаление файла на сервере

Для удаления файла с сервера при использовании прямых запросов к Зениту используется отдельный эндпоинт GET /zenith-object/api/v1/raw/files/delete. В параметре запроса filename передается имя файла в формате Зенита. Например:

```
curl -X 'GET' \
  'http://localhost:18080/zenith-object/api/v1/raw/files/delete?filename=%5C4d334c8b-5d05-4781-b5f7-d831903535b6' \
  -H 'accept: */*' \
  -H 'Authorization: Basic emVuaXRoLWFwaToxMjM0' \
  -H 'X-Zenith-Server: C0'
```

В случае успешного удаления файла возвращается код 200.

5. Описание формата MFO_09_01

Файл этой схемы подаётся, когда необходимо зарегистрировать входящий или исходящий документ.

Формальное описание схемы находится в файле mfo_0901.xsd, который лежит в папке edoscheme Зенит-сервера, возможно кому-то будет удобнее сверяться с этим файлом.

Формат MFO_09_01 создан на основе форматов электронного документооборота (ЭДО) ПАРТАД версии FRD_07_01. При этом данный формат поддерживается фирмой Элдис-Софт и является полностью независимым от своего прародителя; в частности, он никак не зависит от смены версий форматов ПАРТАД.

Процесс регистрации описан выше:

- регистрация входящих документов (см. 3.3.1.1. Регистрация документа в формате Формат `MF0_09_01`), корневой элемент файла `DOCUMENT`;
- формирование автоматических выходных документов (отчётов) (см. 3.4.1.4.1. Формат файла с входными данными), корневой элемент файла `REQUEST_FOR_STATEMENT`;
- регистрация в Зените исходящих, полученных из внешнего источника (см. 3.4.2.1. Регистрация исходящих, полученных из внешнего источника), корневой элемент файла `OUTGOING_DOCUMENT`.

Общим для всех корневых элементов является вложенный элемент `version`, который должен содержать значение `MF0_09_01`. В подразделах описывается структура отдельных элементов.

5.1. Эмитент, ПИФ, управляющая компания

До двух элементов `issuer` задают реестр или группу реестров, к которым относится входящий или исходящий документ.

5.1.1. Без указания реестра

Документ может быть без указания реестра. В случае отчёта это означает, что он формируется по всем реестрам сразу или является отчётом чистого документооборота, в котором данные реестров не используются.

Чтобы сформировать такой документ, не создавайте элементов `issuer`. Отметим, что `<issuer/>` с пустым содержимым вызовет ошибку, элемент должен отсутствовать вовсе.

5.1.2. Конкретный эмитент

Конкретный эмитент задаётся следующим блоком:

```
<issuer>
  <issuer_type>2</issuer_type>
  <issuer_name>
    <party_id>
      <id>NNN</id>
    </party_id>
  </issuer_name>
</issuer>
```

Значение `2` в элементе `issuer_type` определяет, что речь идёт об эмитенте, а `NNN` задаёт код эмитента по справочнику Зенита.

5.1.3. Конкретный ПИФ

Для указания ПИФа требуется два элемента `issuer`, которые задают ПИФ и его УК:

```
<issuer>
  <issuer_type>2</issuer_type>
  <issuer_name>
    <party_id>
      <id>Код ПИФа</id>
    </party_id>
  </issuer_name>
</issuer>
<issuer>
  <issuer_type>1</issuer_type>
  <issuer_name>
    <party_id>
      <id>Код УК</id>
    </party_id>
  </issuer_name>
</issuer>
```

В элементе `issuer`, `issuer_type` которого равен `2` задаётся код ПИФа, в элементе, `issuer_type` которого `1` — код управляющей компании.

УК обязательно должна быть указана и именно та, что соответствует указанному ПИФу.

5.1.4. По всем ПИФам некоторой УК

Чтобы сформировать отчёт по всем ПИФам некоторой УК, создайте один элемент `issuer` с `issuer_type` равным `1`:

```
<issuer>
  <issuer_type>1</issuer_type>
  <issuer_name>
    <party_id>
      <id>Код УК</id>
    </party_id>
  </issuer_name>
</issuer>
```

Такая схема используется только для формирования некоторых отчётов (таких как многореестровый регистрационный журнал). Для регистрации входящего или готового исходящего не применяется никогда.

5.1.5. Формирование отчета по группе реестров

Некоторые отчёты (такие, как поиск ЗЛ по всем реестрам) могут формироваться не только по конкретному эмитенту или ПИФу, но и по их группе.

Для формирования отчёта по группе реестров, введите `issuer` с `issuer_type`, равным `3` и `NNN` — кодом по справочнику "Группы эмитентов"/"Группы ПИФов":

```
<issuer>
  <issuer_type>3</issuer_type>
  <issuer_name>
    <party_id>
      <id>NNN</id>
    </party_id>
  </issuer_name>
</issuer>
```

Эмитент/ПИФ документа в этом случае будет `<Не указан>`.

Выбранная группа реестров заполняется как часть данных поручения.

Такая схема используется только для формирования отчётов, для регистрации входящего или готового исходящего не применяется.

5.2. Номер и дата документа

Во входящем документе можно задать номер и дату документа у отправителя:

```
<header>
  <doc_num>2093-7a</doc_num>

  <doc_date>
    <date>2009-03-08</date>
```



```
<!-- альтернатива: -->
<datetime>2009-03-08T12:30:00</datetime>
</doc_date>
</header>
```

Поля `date` и `datetime` задают дату документа и являются взаимоисключающими. Дату допустимо задавать в любом (но только одном) из них. Время в поле `datetime` игнорируется.

Наличие элементов с номером и датой документа обязательно. Если требуется зарегистрировать документ без указания номера, в `doc_num` нужно записать специальное значение `UKWN`. Если требуется оставить не заполненной (нулевой) дату, пишите в поле `doc_date.date` специальное значение `1900-01-01`.

5.3. Отправитель входящего документа

По умолчанию отправителем для документов подсистемы ПИФ является управляющая компания, а для документов подсистемы СВР отправитель по умолчанию не определён (тип "<Не указан>", все поля пустые).

Используя элемент `sender` можно задать собственные данные отправителя.

```
<sender>
  <party_type>1</party_type>
  <party_id><id>3</id></party_id>
  <account_type>3</account_type>
  <individual_or_entity>2</individual_or_entity>
  <name>Иванов Иван Иванович</name>
  <short_name>ИВАНОВ И.И.</short_name>
  <post_address>
    <plain>
      Новосибирская обл., Сузунский1 р-н, г. Сузун1, пер. Сузунский1, д.2, кв.(оф.)19
    </plain>
  </post_address>
  <individual_document>
    <doc_type>
      <individual_document_type_code>21</individual_...>
    </doc_type>
    <doc_ser>50 00</doc_ser>
    <doc_num>191345</doc_num>
    <doc_date>1980-05-05</doc_date>
    <org>Центральным РОВД г.Новосибирска</org>
  </individual_document>
</sender>
```

В бланке документа это будет выглядеть следующим образом

Элемент `party_type` содержит код типа отправителя, заполняемый по справочнику Зенита "Тип отправителя".

Код типа отправителя	Расшифровка	Значение поля <code>party_id.id</code>
1	Зарегистрированное лицо	Номер счёта ЗЛ в реестре
2	Эмитент	игнорируется
3	Регистратор	игнорируется
10000	Управляющая компания	игнорируется
прочие	-	Номер клиента (гиперсчёта)

Элемент `account_type` используется только для отправителя типа "Зарегистрированное лицо" и содержит код типа счёта, кодируемый по справочнику "Тип счёта".

Элемент `individual_or_entity` определяет, юридическим (значение 1) или физическим лицом (значение 2) является отправитель.

Элемент `individual_document_type_code` задаёт тип документа, удостоверяющего личность ФЛ и содержит код по справочнику Зенита "Документы-удостоверения ФЛ".

Допустимо заполнение не всех полей отправителя. Если пустыми оставлены все элементы, кроме `party_type` и `party_id.id` (для тех типов, для которых он имеет смысл), то прочие поля автоматически заполняются из базы данных: для ЗЛ — данными счёта из реестра, для эмитента — данными из карточки эмитента и т.п.

В примере выше приведено сообщения с отправителем-ФЛ. Отправитель-ЮЛ заполняется почти так же, за исключением того, что элемент `individual_document`, содержащий информацию о документе физ.лица заменяется на элемент `entity_reg_dtls` с регистрационными данными юр.лица:

```
<sender>
  <party_type>1</party_type>
  <party_id><id>5</id></party_id>
  <account_type>3</account_type>
  <individual_or_entity>1</individual_or_entity>
  <name>Закрытое Акционерное общество "Новосибирский молочный комбинат"</name>
  <short_name>ЗАО "НМК"</short_name>
  <post_address>
    <plain>
      630089, Новосибирская обл., г. Новосибирск, ул. 1 мая, д.12
    </plain>
  </post_address>
  <entity_reg_dtls>
    <reg_doc_type>
      <entity_reg_doc_type_code>1</entity_reg_doc_type_code>
    </reg_doc_type>
    <reg_num>13-12-1998</reg_num>
    <date_of_incorporation>1998-11-22</date_of_incorporation>
    <reg_org>Регистрационная палата г.Новосибирска</reg_org>
  </entity_reg_dtls>
</sender>
```

Элемент `entity_reg_doc_type_code` кодирует тип регистрационного документа по справочнику "Документы гос.регистрации ЮЛ".

5.4. Доставка входящего документа и отправка ответных исходящих

Эти элементы используются при регистрации входящих и исходящих документов. Способ доставки, место приёма и место выдачи можно задать элементами `bearer` и `recipient`:

```
<bearer>
  <!-- способ доставки входящего -->
  <way>NNN</way>

  <!-- место приёма (может отсутствовать) -->
  <abonent>SSS</abonent>
</bearer>
<recipient>
  <!-- способ отправки исходящего -->
  <way>NNN</way>

  <!-- место выдачи (может отсутствовать) -->
  <abonent>SSS</abonent>
</recipient>
```

, где `NNN` — код по пользовательскому справочнику "Способы доставки документа", а `SSS` — код места приёма и места выдачи, который может быть кодом подразделения, трансфер-агента или абонента ЭДО, см. в Зенит-клиенте режим "Территориальная инфраструктура".

В исходящем документе можно использовать только `recipient`.

Элементы `bearer` и `recipient` могут отсутствовать целиком или содержать только `way`, отсутствующие поля заполняются значениями по умолчанию.

5.4.1. Значение по умолчанию: способ доставки

Способом доставки входящего (`bearer/way`) по умолчанию считается "Через ZenithObj".

Способ отправки исходящего (`recipient/way`) по умолчанию такой же как способ доставки, а если тот не указан, то "Через ZenithObj".

5.4.2. Значение по умолчанию: место приема и место выдачи

В исходящем документе место выдачи по умолчанию — текущее подразделение.

Для входящих документов заполнение мест приёма и выдачи по умолчанию различается для СВР и ПИФ.

У документов СВР или "чистого" документооборота местом приёма по умолчанию считается текущее подразделение. Место выдачи по умолчанию совпадает с местом приёма.

Таким образом, если нужно указать нестандартное место, достаточно заполнить `bearer/abonent`, а `recipient/abonent` можно не указывать, он автоматически станет таким же.

В документах подсистемы ПИФ может быть задан агент УК или пункт приёма заявок:

```
<agent_point_name>
  <point_name>
    <id>PPP</id>
  </point_name>
  <agent_name>
    <id>AAA</id>
  </agent_name>
</agent_point_name>
```

, где `PPP` и `AAA` – коды по справочнику Зенита для пункта приёма заявок и агента УК соответственно. Любой из кодов может отсутствовать. Местом приёма документа считается первое, что задано из следующего списка:

1. пункт приёма заявок
2. агент УК
3. то, что указано в `bearer/abonent`
4. управляющая компания (задаётся элементом `issuer`, см. 5.1. Эмитент, ПИФ, управляющая компания)

Если место приёма не задано ни одним из способов, будет ошибка регистрации документа.

Во избежание неоднозначностей, если заполнен ППЗ или агент УК, элемент `bearer/abonent` должен отсутствовать.

Местом выдачи исходящих документов по ПИФам по умолчанию всегда считается УК (даже если заполнен агент, пункт приёма или `bearer/abonent`). Если УК не задана и отсутствует `recipient/abonent`, место выдачи остаётся не указанным.

5.5. Тип носителя

Применяется только при регистрации входящего документа. По умолчанию тип носителя — электронный документ.

Его можно переопределить элементом `media`:

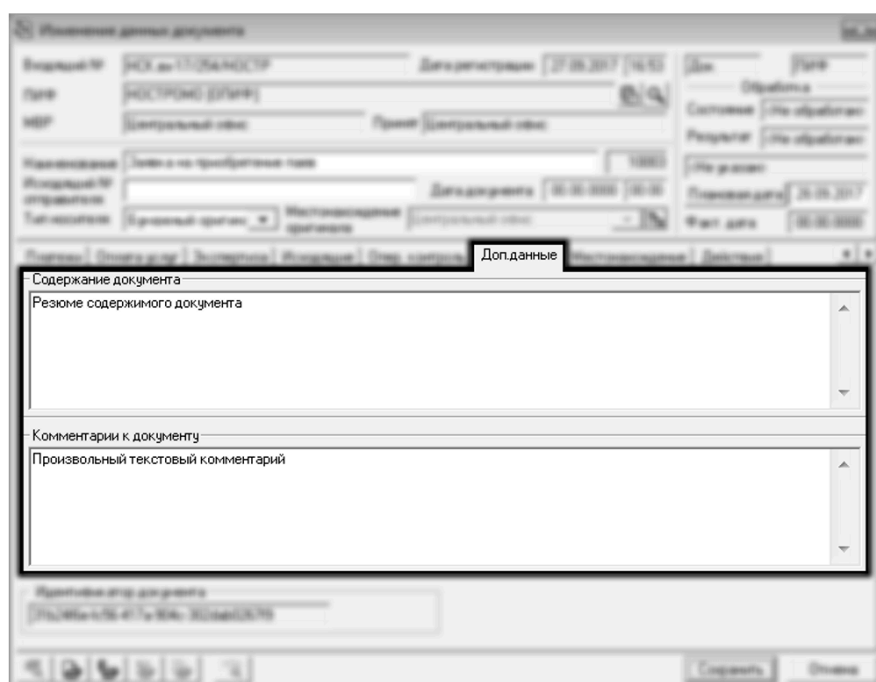
```
<media>
  <id>NNN</id>
</media>
```

, где **NNN** — код по пользовательскому справочнику "Тип носителя".

5.6. Комментарий и содержимое документа

Два простых текстовых поля:

```
<comment>Произвольный текстовый комментарий</comment>
<content>Резюме содержимого документа</content>
```

The image is a screenshot of a web-based document management system. It features a form titled "Исходные данные документа" (Document Source Data). The form contains several input fields for document metadata, including "Исходный №" (Original No.), "Дата регистрации" (Registration Date), "Тип" (Type), "Состояние" (Status), "Результат" (Result), "Имя файла" (File Name), "Печенька" (Cookie), "Печенька дата" (Cookie Date), and "Файл" (File). Below the main form, there are two sections: "Содержание документа" (Document Content) and "Комментарии к документу" (Comments to the Document). The "Содержание документа" section contains a text area with the text "Резюме содержимого документа" (Summary of document content). The "Комментарии к документу" section contains a text area with the text "Произвольный текстовый комментарий" (Arbitrary text comment). The interface also includes a "Дополнительные" (Additional) tab and a "Справка" (Help) button.

Любой элемент может отсутствовать или быть пустым. Для исходящих может заполняться только `<comment>`.

5.7. Тип формируемого отчета

Код типового исходящего задаётся следующим элементом:

```
<statement_type>NNN</statement_type>
```

, где **NNN** — код по справочнику типов выходных документов. Например, если **NNN** содержит код 18, будет введён следующий отчёт:

Изменение данных документа

Исходный №: 40-00-00000004 Дата регистрации: 20.01.2009 Дик: [] Тип: []

Заголовок: [] Состояние: [] Обработка: []

МРР: [] Прием: [] Прием: [] Прием: [] Прием: [] Прием: [] Прием: [] Прием: []

Наименование: [] Дата документа: 20.01.2009 Плановая дата: 20.01.2009 Факт дата: 20.01.2009

Исходный №: [] Тип документа: [] Тип документа: [] Тип документа: [] Тип документа: [] Тип документа: [] Тип документа: [] Тип документа: []

Оформить: [] Приложить: [] Приложить: [] Приложить: [] Приложить: [] Приложить: [] Приложить: [] Приложить: []

Ссылки на этот документ:

№	Тип	Статус	Зарегистрированное ...	Контрагент
1	Справка об операциях по счету (18)		ИВАНОВ И.И.	

Сохранить Отмена

5.8. Пользовательское наименование отчета

Может задаваться только при регистрации исходящих документов.

По умолчанию наименование регистрируемого исходящего совпадает с наименованием типового. Если вы хотите задать особое наименование, укажите его в элементе `statement_title`:

```
<statement_title>Отчет о счетах ЗЛ через Зенит-API</statement_title>
```

В Зените это будет выглядеть вот так:

5.9. Присвоение исходящего номера

Исходящие документы могут формироваться как с присвоением исходящего номера (для официальной выдачи), так и без его присвоения (для собственных нужд организации).

Признак присвоения номера задаётся следующим элементом:

```
<assign_outnum>1</assign_outnum>
```

Единица — присваивать номер, ноль — формировать без присвоения исходящего номера. Элемент может отсутствовать, в этом случае номер не присваивается.

5.10. Даты отчета

Некоторые отчёты формируются на конкретную дату (например, выписка со счёта), другие же требуют указания диапазона дат (например, справка об операциях), третьи не предусматривают указания дат (например, поиск ЗЛ по всем реестрам).

5.10.1. Отчеты без указания дат

В таких отчётах укажите элемент `statement` без содержимого или опустите его вовсе:

```
<statement/>
```

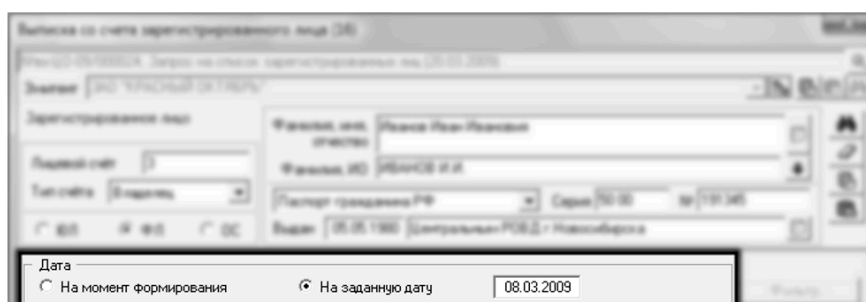
В Зенит-клиенте это будет выглядеть вот так:



5.10.2. Отчеты с одной датой

Дата задаётся следующим блоком:

```
<statement>
  <date>2009-03-08</date>
</statement>
```



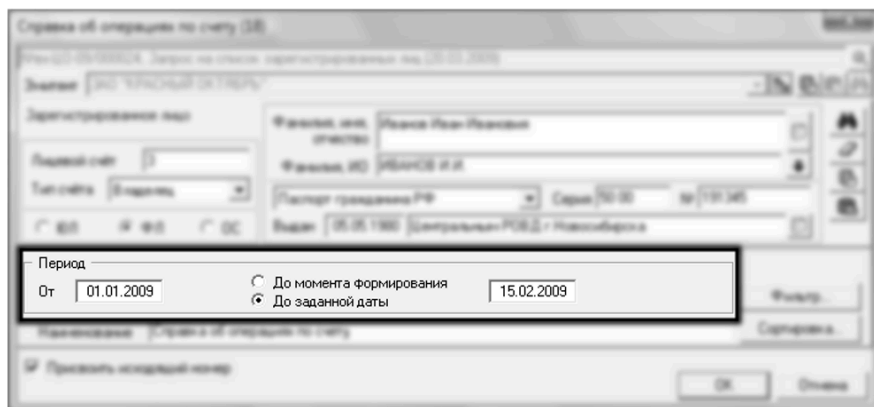
Если дата совпадает с текущим календарным днём, время, на которое формируется отчёт, будет совпадать с текущим временем по часам сервера. Если указана старая дата, отчёт формируется на конец дня (время `24:00`).

Для формирования отчёта без указания даты ("на момент формирования"), задайте элемент `<statement>` без содержимого или опустите его вовсе.

5.10.3. Отчеты с диапазоном дат

Диапазон дат задаётся следующим блоком:

```
<statement>
  <start_date>2009-01-01</start_date>
  <end_date>2009-02-15</end_date>
</statement>
```



Если `end_date` совпадает с текущим календарным днём, то время окончания диапазона будет совпадать с текущим временем по часам сервера, в противном случае будет указано `24:00`.

Чтобы сформировать отчёт без указания даты "до" (т.е. "до момента формирования"), просто опустите элемент `<end_date>`:

```
<statement>
  <start_date>2009-01-01</start_date>
</statement>
```

Дата начала периода `<start_date>` так же может быть опущена, что означает формировать "от начала времён". В Зените это будет представлено нулевой датой `00.00.0000`.

5.11. Зарегистрированное лицо

Многие отчёты выполняются по конкретному счёту зарегистрированного лица: выписка со счёта, справка об операциях и другие. ЗЛ описывается следующим блоком:

```
<account_dtls>
  <account_id>
    <id>NNN</id>
  </account_id>
</account_dtls>
```

Так же допустимо указывать альтернативный блок:

```

<account_holder>
  <party_id>
    <id>NNN</id>
  </party_id>
</account_holder>

```

, где **NNN** — номер счёта в реестре. По историческим причинам допустимы два альтернативных способа задания номера счёта, но в каждом конкретном файле должен использоваться только один из них.

Идентификатор ЗЛ заполняется данными реестра, на основании номера счёта. Если счёт с указанным номером в реестре отсутствует, будет выдана ошибка.

5.12. Операция

При запросе уведомления об операции заполняется ссылка на операцию. Можно указывать номер как по технологическому, так и по регистрационному журналу:

```

<operation>
  <oper_number>204</oper_number> <!-- по рег. журналу -->
</operation>

```

или

```

<operation>
  <tech_number>815</tech_number> <!-- по тех. журналу -->
</operation>

```

5.13. Фильтр

Дополнительная фильтрация, доступная в поручении по кнопке "Фильтр" задаётся в следующем блоке:

```
<add_info>
  <filter>
    <!-- здесь данные фильтра -->
  </filter>
</add_info>
```

Формат данных фильтра зависит от выбранного базового отчёта. Формальное описание форматов приведено в файле `zenith_filters.xsd`, являющимся частью документации к Зениту. Просто для примера можно привести вариант фильтра к "Отчёту об изменении анкетных данных":

```
<add_info>
  <filter>
    <CHANGE>
      <TYPE>1</TYPE>
      <NAME>1</NAME>
      <DOC>1</DOC>
      <CITIZEN>0</CITIZEN>
      <ADDR_POST>0</ADDR_POST>
      <BANK>0</BANK>
      <OTHER>0</OTHER>
      <ANALITIC>0</ANALITIC>
      <OFFICIAL>0</OFFICIAL>
      <ADDR_UR>1</ADDR_UR>
      <CHANGE_ONLY>1</CHANGE_ONLY>
    </CHANGE>
  </filter>
</add_info>
```

Ему соответствует следующий бланк фильтра:

5.14. Приложения

Приложение к документу добавляется с помощью блока:

```
<appendix>
  <type>4</type>
  <name>Тестовое наименование документа</name>
  <regdate>2021-11-25</regdate>
  <media>1</media>
  <outnum>1</outnum>
  <docdate>2021-11-25</docdate>
</appendix>
```

Ему соответствует бланк приложения:

Элементы внутри `appendix` содержат следующую информацию:

- `type` - код типа документа приложения по справочнику типов документа,

- `name` - наименование приложения,
- `regdate` - дату регистрации,
- `media` - код типа носителя по справочнику типов носителя,
- `outnum` - исходящий № отправителя,
- `docdate` - дата документа отправителя.

Чтобы прикрепить несколько приложений, указывайте подряд несколько элементов `appendix`.